

Intitulé de la matière : INITIATION A UML

Objectifs de l'enseignement :

Présenter les éléments de spécification du langage UML et son usage dans le processus de développement logiciel

Connaissances préalables recommandées : Connaissances générales en informatique

Contenu de la matière :

- ✓ Diagrammes structurels ou statiques
 - Diagramme de classes
 - Diagramme d'Objets
 - Diagramme de composants
 - Diagramme de déploiement
 - Diagramme de paquetages
 - Diagramme de structure composite
- ✓ Diagrammes comportementaux ou dynamique
 - Diagramme de cas d'utilisation
 - Diagramme d'activités
 - Diagramme d'états-transitions
 - Diagrammes d'interaction
 - Diagramme de séquence
 - Diagramme de communication
 - Diagramme global d'interaction
 - Diagramme de temps
- ✓ Processus de développement (Rational Unified Process)

CHAPITRE I : PRESENTATION UML

I- Généralité

1. Définition

UML (Unified Modeling Language) est un langage unifié pour la modélisation objet.

UML est un langage de modélisation objet et propose donc une notation et une sémantique associée à cette notation : **des modèles**.

UML n'est pas un processus. Il ne constitue pas une démarche proposant un enchaînement d'étapes et d'activités qui mènent à la résolution d'un problème posé : **UML n'est donc pas une méthode**.

UML a une approche entièrement objet. Il couvre tout le cycle de développement d'un logiciel : **analyse, conception et réalisation**

Uml est non fonctionnel : le système est décomposé en objets collaborant, plutôt qu'en tâches décomposées en fonctions plus simples à réaliser.

2. Historique

UML unifie des méthodes objet, et plus particulièrement les méthodes Booch'93 de G. Booch, OMT-2 (Object Modeling Technique) de J. Rumbaugh et OOSE (Object-Oriented Software Engineering) d'I. Jacobson.

UML reprend en particulier les notions de partitions en sous-systèmes de Booch'93, de classes et d'associations d'OMT-2, et d'expression des besoins par les interactions entre les utilisateurs et le système d'OOSE.

Les grandes étapes de la diffusion d'UML peuvent se résumer comme suit :

- 1994-1996 : rapprochement des méthodes OMT, BOOCH et OOSE et naissance de la première version d'UML ;
- 1997 : version 1.1 d'UML adoptée par l'Object Management Group (OMG) ;
- 1998-1999 : sortie des versions 1.2 à 1.3 d'UML ;
- 2000-2001 : sortie des dernières versions suivantes 1.x ;

- 2002-2003 : préparation de la V2 ;
- 2004 : sortie de la V2.1 ;
- 2007 : sortie de la V2.1.1 ;
- Depuis 2015 : UML 2.5.

II- Concepts de l'approche objet

Aujourd'hui l'approche objet occupe une place prépondérante dans le génie logiciel. En effet, ces dernières années nous avons assisté tout d'abord à une utilisation plus large des langages de programmation objet de référence comme C++, C#, Java, python et ensuite à l'introduction des concepts objet dans d'autres langages comme par exemple VB.NET, Perl et même Cobol. . Le développement très important d'applications liées à Internet constitue une des principales explications de ce phénomène.

. Le développement très important d'applications liées à Internet constitue une des principales explications de ce phénomène.

Notre objectif est donc de présenter l'essentiel des concepts objet qui nous paraissent nécessaires à une bonne compréhension d'UML.

Les concepts qui nous semblent importants à bien maîtriser sont les suivants :

- objet et classe,
- encapsulation et interface,
- association et agrégation de classes,
- généralisation et spécialisation de classe,
- polymorphisme,
- persistance.

1. Objet et classe

Concept d'objet

Un **objet** représente une entité du monde réel (ou du monde virtuel pour les objets immatériels) qui se caractérise par un ensemble de propriétés (attributs), des états significatifs et un comportement.

L'**état** d'un objet correspond aux valeurs de tous ses attributs à un instant donné. Les propriétés sont définies dans la classe d'appartenance de l'objet.

Le **comportement** d'un objet est caractérisé par l'ensemble des opérations qu'il peut exécuter en réaction aux messages provenant des autres objets. Les opérations sont définies dans la classe d'appartenance de l'objet.

Exemple :

Considérons l'employé Durand, n° 1245, embauché en tant qu'ingénieur travaillant sur le site N.

Cet objet est caractérisé par la liste de ses **attributs** et son **état** est représenté par les valeurs de ses attributs :

- n° employé : 1245,
- nom : Durand,
- qualification : ingénieur,
- lieu de travail : site N.

Son **comportement** est caractérisé par les opérations qu'il peut exécuter. Dans notre cas nous pouvons avoir les opérations suivantes :

- entrer dans l'organisme,
- changer de qualification,
- changer de lieu de travail,
- sortir de l'organisme.

Concept de classe

Une classe est l'abstraction d'un ensemble d'objets qui possèdent une structure identique (liste des attributs) et un même comportement (liste des opérations).

Un objet est une **instance** d'une et une seule classe.

Une **classe abstraite** est une classe qui n'a pas d'instance.

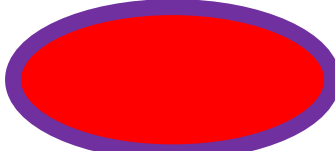
Exemple

Considérons la classe Employé qui représente l'ensemble des employés d'une entreprise. La description de la classe Employé comportera les éléments suivants :

- Nom de classe : Employé.
- Attributs :
 - numéro,
 - nom,
 - qualification,
 - site de travail.
- Opérations :
 - engager un employé,
 - consulter un employé,
 - modifier un employé,
 - licencier un employé.

Exercice 1:

Dans cet exercice, on vous représente les objets suivants :



- 1) Donner le nom d'une classe ayant permis de représenter ces objets.
- 2) Définissez ses attributs ainsi que ses opérations
- 3) Identifier le nom de chaque objet
- 4) Pour chaque objet, donner les valeurs de ses attributs

- 5) Quels sont les objets qui se distinguent des autres ?
- 6) Proposer des attributs spécifiques à ces objet énumérés.

2. Encapsulation et interface

L'approche objet se caractérise par le regroupement dans une même classe de la description de la structure des attributs et de la description des opérations. Ce regroupement des deux descriptions porte le nom **d'encapsulation données-traitements**.

Plus précisément, les données ne sont accessibles qu'à partir d'opérations définies dans la classe.

Le principe d'encapsulation renforce l'autonomie et l'indépendance de chaque classe et donne une potentialité accrue de définition de classe réutilisable.

L'ensemble des opérations d'une classe rendu visible aux autres classes porte le nom d'interface. Le schéma d'ensemble du principe de l'encapsulation est présenté à la figure 1.

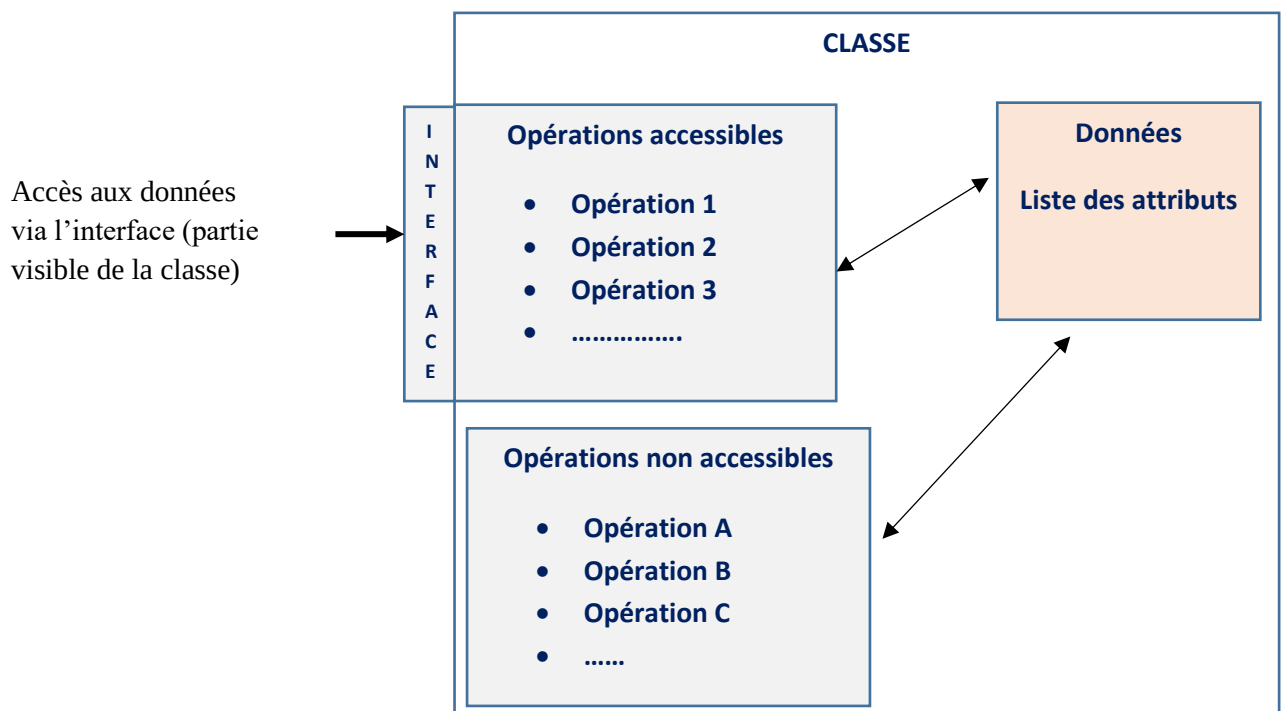


Figure 1: Schéma de principe de l'encapsulation

L'**association** représente une relation entre plusieurs classes.

Elle correspond à l'abstraction des liens qui existent entre les objets dans le monde réel.

Les multiplicités (ou cardinalités) et les rôles des objets participant aux relations complètent la description d'une association.

Les exemples d'associations sont donnés directement dans les diagrammes de classe d'UML.

L'**agrégation** est une forme particulière d'association entre plusieurs classes.

Elle exprime le fait qu'une classe est composée d'une ou plusieurs autres classes.

La relation composant-composé ou la relation structurelle représentant l'organigramme d'une entreprise sont des exemples types de la relation d'agrégation.

3. Généralisation et spécialisation de classe

La **généralisation** de classes consiste à factoriser dans une classe, appelée super-classe, les attributs et/ou opérations des classes considérées. Appliquée à l'ensemble des classes, elle permet de réaliser une hiérarchie des classes.

La **spécialisation** représente la démarche inverse de la généralisation puisqu'elle consiste à créer à partir d'une classe, plusieurs classes spécialisées.

Chaque nouvelle classe créée est dite spécialisée puisqu'elle comporte en plus des attributs ou opérations de la **super-classe** (disponibles par héritage) des attributs ou opérations qui lui sont propres.

Une classe spécialisée porte aussi le nom de **sous-classe**. La spécialisation de classe se construit en deux temps : d'abord par héritage des opérations et des attributs d'une super-classe et ensuite par ajout d'opérations et/ou d'attributs spécifiques à la sous-classe.

La **généralisation-spécialisation** est l'un des mécanismes les plus importants de l'approche objet qui facilite la réutilisation des classes.

4. Polymorphisme

Le polymorphisme est la capacité donnée à une même opération de s'exécuter différemment suivant le contexte de la classe où elle se trouve.

Ainsi une opération définie dans une super-classe peut s'exécuter de manière différente selon la sous-classe où elle est héritée.

En fait lors de l'exécution, l'appel de l'opération va automatiquement déclencher l'exécution de l'opération de la sous-classe concernée. Dans le déroulement de l'exécution de l'opération de la sous-classe, il est possible de faire appel à l'opération de la super-classe qui contient en général la partie commune applicable aux deux sousclasses.

Exemple

Soit la classe Employé et ses deux sous-classes **Cadre** et **NonCadre**.

- Nom de classe : Employé.
 - Attributs :
 - numéro,
 - nom,
 - salaire de base.
 - Opérations : calculSalaire().
- Nom de la sous-classe : Cadre.
 - Attributs : niveau d'encadrement.
 - Opérations : calculSalaire().
- Nom de la sous-classe : NonCadre.
 - Attributs : niveau de spécialisation.
 - Opérations : calculSalaire().

Le principe de calcul du salaire est de calculer pour chaque type d'employé une prime spécifique en fonction soit du niveau d'encadrement, soit du niveau de spécialisation selon le type d'employé.

Dans cet exemple, lorsque l'on appelle l'opération calculSalaire(), c'est l'opération de sous-classe Cadre ou celle de la sous-classe NonCadre qui est en fait activée selon l'objet concerné.

L'opération de la sous-classe fait en général appel explicitement à l'opération calculSalaire() de la super-classe pour bénéficier des traitements communs aux cadres et non cadres et ensuite il y aura poursuite du traitement spécifique à la sous-classe.

5. Persistance

La persistance est la propriété donnée à un objet de continuer à exister après la fin de l'exécution du programme qui l'a créé.

Par défaut dans l'approche objet, aucun objet n'est persistant. Les modèles décrivent le système en exécution en mémoire centrale et ne tiennent pas compte a priori de l'état du système qui doit être stocké sur disque.

La gestion de la mémoire incombe au programmeur avec notamment le problème de la libération des espaces.

6. Avantages du développement à l'aide des langages objet

Par rapport à une approche fonctionnelle associée à un développement classique mené à l'aide de langages procéduraux, on est en droit de s'interroger sur les avantages qu'apporte réellement le développement à l'aide d'un langage objet comme par exemple C++, C# ou Java.

En fait, deux avantages prépondérants sont mis en général en avant lorsque l'on choisit une approche objet :

La modularité : Par construction, étant donné que l'on conçoit des classes représentant une entité de taille limitée en données et en opérations, il est plus aisé de construire des systèmes modulables que si l'on élabore une seule base de données d'une part et un seul logiciel d'autre part.

La réutilisabilité : La définition d'un système à l'aide de classe ayant chacune la responsabilité d'un sous-ensemble de données et des opérations associées favorise fortement la potentialité de trouver des classes réutilisables. La réutilisation de classe se réalise soit sur le plan métier à l'intérieur d'une même entreprise dans

des applications différentes, soit sur le plan technique à l'échelle de tous les développements réalisés à l'aide d'un même langage. Sur ce dernier aspect, c'est toute l'approche du développement par composant qui est en jeu.

Au-delà de ces deux avantages majeurs, la maintenance élémentaire de chaque classe est en soi plus simple à réaliser que celle d'un logiciel unique traitant toutes les données d'un système.

NB : Il importe bien entendu dans l'approche objet de construire son système en veillant à minimiser le nombre de relations entre classes.

III- Notions importantes d'UML

1. Règles générales

Les principaux éléments généraux d'UML que nous présenterons sont : le méta-modèle, le stéréotype, la valeur marquée, la note, la contrainte, et la relation de dépendance.

a. Le méta-modèle

Le langage de modélisation UML respecte un certain nombre de règles sur les concepts manipulés (classes, attributs, opérations, paquetages...) ainsi que sur la syntaxe d'écriture et le formalisme de représentation graphique.

L'ensemble de ces règles constitue en soi un langage de modélisation qui a fait l'objet d'un méta-modèle UML.

L'intérêt de disposer d'un méta-modèle UML permet de bien maîtriser la structure d'UML et de faciliter son évolution.

Le méta-modèle d'UML est complètement décrit dans la norme d'UML.

b. le stéréotype

Un stéréotype constitue un moyen de classer les éléments de la modélisation.

Un certain nombre de stéréotypes sont déjà définis dans UML mais d'autres valeurs de stéréotypes peuvent être ajoutées si cela est nécessaire soit à l'évolution

générale d'UML, soit à la prise en compte de situations particulières propres aux entreprises.

Les stéréotypes peuvent s'appliquer à n'importe quel concept d'UML. Nous nous intéresserons dans ce cours à certains d'entre eux que nous présenterons au niveau des diagrammes lorsque leur utilisation nous paraîtra pertinente.

En particulier, dans le diagramme de classe, le stéréotype permet de considérer de nouveaux types de classe. Cette possibilité d'extension pour les classes, se définit donc au niveau méta-classe.

Formalisme et exemple

Le nom du stéréotype est indiqué entre guillemets.

Un acteur peut être vu comme un stéréotype particulier d'une classe appelée acteur. L'exemple (fig. 1) montre une classe Client stéréotypée comme « acteur ».

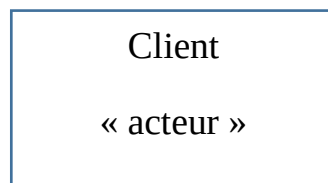


Figure 2: Exemple d'une classe stéréotypée

c. La valeur marquée

UML permet d'indiquer des valeurs particulières au niveau des éléments de modélisation et en particulier pour les attributs de classe. Une valeur marquée se définit au niveau méta-attribut.

Formalisme et exemple

La valeur marquée est mise entre accolades avec indication du nom et de la valeur : {persistance : string} si l'on veut ajouter ce type d'attribut dans une classe.

Afin de donner la possibilité de spécialiser chaque application d'UML à son propre contexte, UML propose de définir un profil d'utilisation caractérisé principalement par la liste des stéréotypes, la liste des valeurs marquées et les contraintes spécifiées pour un projet donné

d. La note

Une note correspond à un commentaire explicatif d'un élément d'UML.

Formalisme et exemple

La figure 3 montre le formalisme et un exemple de la représentation d'une note.

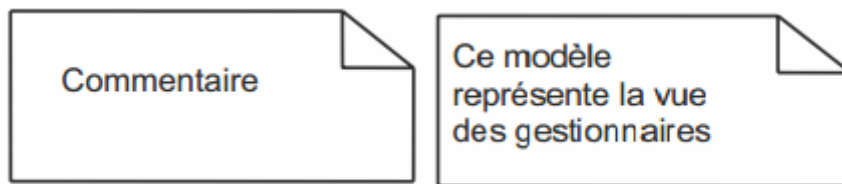


Figure 3: Formalise d'utilisation d'une note

e. La contrainte

Une contrainte est une note ayant une valeur sémantique particulière pour un élément de la modélisation.

Une contrainte s'écrit entre accolades {}.

Dans le cas où la contrainte concerne deux classes ou plus, celle-ci s'inscrit à l'intérieur d'une note.

Formalisme et exemple

- Première forme d'écriture d'une contrainte : {ceci est une contrainte}.
- Deuxième forme d'écriture : à l'intérieur d'une note (fig. 4).

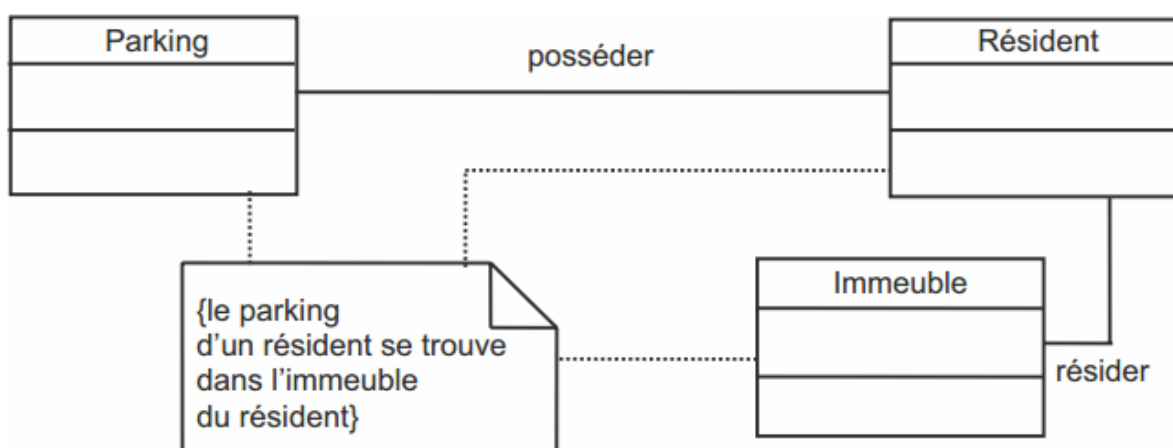


Figure 4: Utilisation d'une contrainte

2. Présentation des diagrammes

UML dans sa version 2 propose treize diagrammes qui peuvent être utilisés dans la description d'un système. Ces diagrammes sont regroupés dans deux grands ensembles.

a. Les diagrammes structurels

Ces diagrammes, au nombre de six, ont vocation à représenter l'aspect statique d'un système (classes, objets, composants...).

Diagramme de classe :

Ce diagramme représente la description statique du système en intégrant dans chaque classe la partie dédiée aux données et celle consacrée aux traitements. C'est le diagramme pivot de l'ensemble de la modélisation d'un système.

Diagramme d'objet :

Le diagramme d'objet permet la représentation d'instances des classes et des liens entre instances.

Diagramme de composant :

Ce diagramme représente les différents constituants du logiciel au niveau de l'implémentation d'un système.

Diagramme de déploiement :

Ce diagramme décrit l'architecture technique d'un système avec une vue centrée sur la répartition des composants dans la configuration d'exploitation.

Diagramme de paquetage :

Ce diagramme donne une vue d'ensemble du système structuré en paquetage. Chaque paquetage représente un ensemble homogène d'éléments du système (classes, composants...).

Diagramme de structure composite :

Ce diagramme permet de décrire la structure interne d'un ensemble complexe composé par exemple de classes ou d'objets et de composants techniques. Ce diagramme met aussi l'accent sur les liens entre les sous-ensembles qui collaborent.

b. Les diagrammes de comportement

Ces diagrammes représentent la partie dynamique d'un système réagissant aux événements et permettant de produire les résultats attendus par les utilisateurs. Sept diagrammes sont proposés par UML :

Diagramme des cas d'utilisation :

Ce diagramme est destiné à représenter les besoins des utilisateurs par rapport au système. Il constitue un des diagrammes les plus structurants dans l'analyse d'un système.

Diagramme d'état-transition (machine d'état) :

Ce diagramme montre les différents états des objets en réaction aux événements.

Diagramme d'activités :

Ce diagramme donne une vision des enchaînements des activités propres à une opération ou à un cas d'utilisation. Il permet aussi de représenter les flots de contrôle et les flots de données.

Diagramme de séquence :

Ce diagramme permet de décrire les scénarios de chaque cas d'utilisation en mettant l'accent sur la chronologie des opérations en interaction avec les objets.

Diagramme de communication :

Ce diagramme est une autre représentation des scénarios des cas d'utilisation qui met plus l'accent sur les objets et les messages échangés.

Diagramme global d'interaction :

Ce diagramme fournit une vue générale des interactions décrites dans le diagramme de séquence et des flots de contrôle décrits dans le diagramme d'activités.

Diagramme de temps :

Ce diagramme permet de représenter les états et les interactions d'objets dans un contexte où le temps a une forte influence sur le comportement du système à gérer.

Aujourd'hui UML 2 décrit les concepts et le formalisme de ces treize diagrammes mais ne propose pas de démarche de construction couvrant l'analyse et la

conception d'un système. Ce qui a pour conséquence par exemple de ne pas disposer d'une vision des interactions entre les diagrammes.

Formalisme et exemple

Afin de donner un premier aperçu des principaux diagrammes tant sur l'aspect du formalisme que sur leur usage, nous proposons à titre introductif un petit exemple très simple.

Considérons une nouvelle société de formation qui souhaite développer un premier niveau de site web dans lequel elle présente succinctement les formations proposées et enregistre en ligne les demandes de catalogue.

Nous pouvons dès ce stade de l'analyse représenter le diagramme des cas d'utilisation (fig. 5).

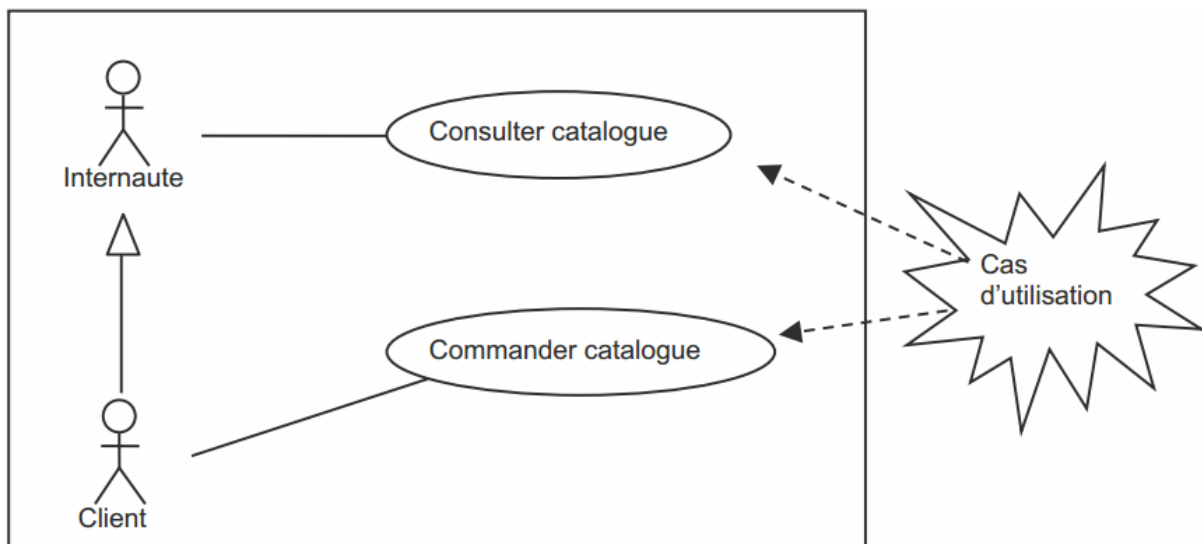


Figure 5: Exemple de diagramme des cas d'utilisation

Le diagramme de classe (fig. 6) va nous permettre de décrire les concepts manipulés, à savoir : Client, Catalogue et Formation.

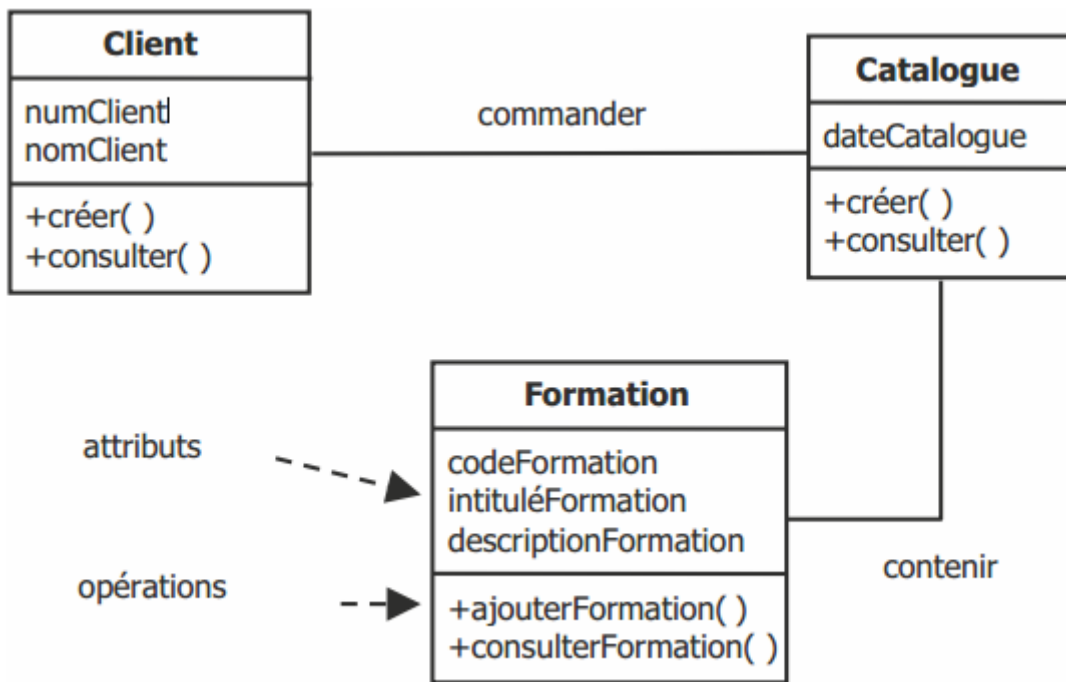


Figure 6: Exemple de diagramme de classe

Le diagramme de séquence va nous permettre de décrire les scénarios des cas d'utilisation du diagramme des cas d'utilisation. À titre d'exemple nous montrons (fig. 7) le scénario correspondant à la consultation du catalogue.

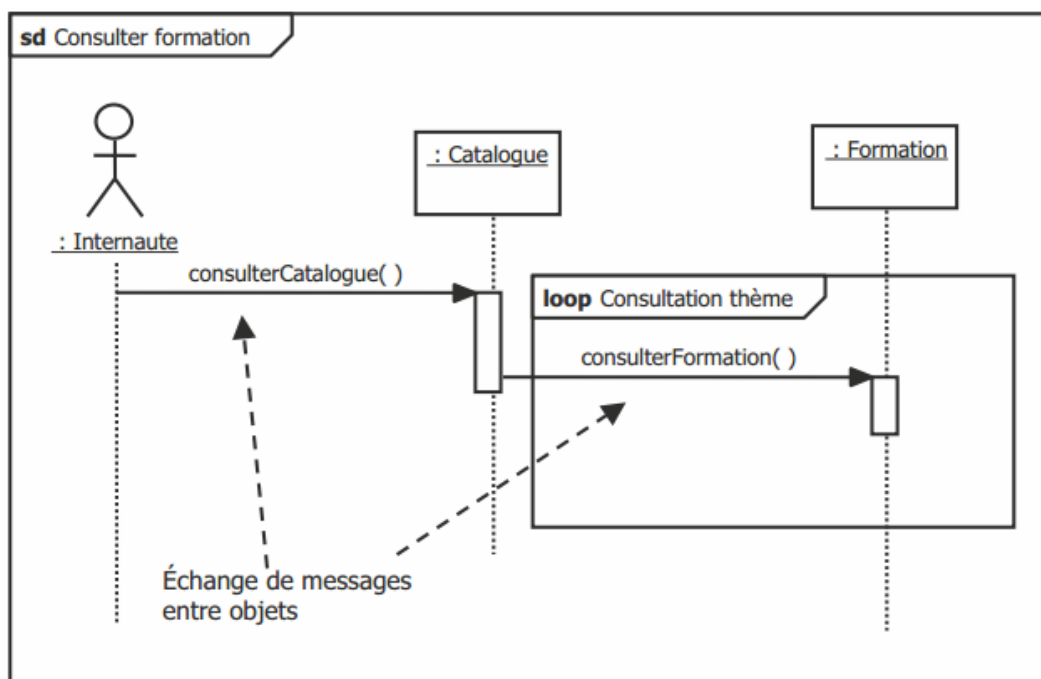


Figure 7: Exemple de diagramme de séquence

Cette première illustration de trois diagrammes donne déjà un éclairage sur les concepts importants que sont la classe, le cas d'utilisation et l'objet.

3. Schéma des diagrammes d'UML

Afin de donner quelques points de repères sur le positionnement et les liens entre tous les diagrammes d'UML, nous donnons ici notre propre vision en proposant un regroupement des diagrammes en quatre ensembles suivant leur finalité :

- **description du système** : huit diagrammes ;
- **architecture technique** : deux diagrammes ;
- **vues globales ou spécialisées** : deux diagrammes ;
- **partition d'éléments de la modélisation** : un diagramme.

Le schéma proposé reprend les treize diagrammes en les répartissant sur les quatre ensembles définis (fig. 8).

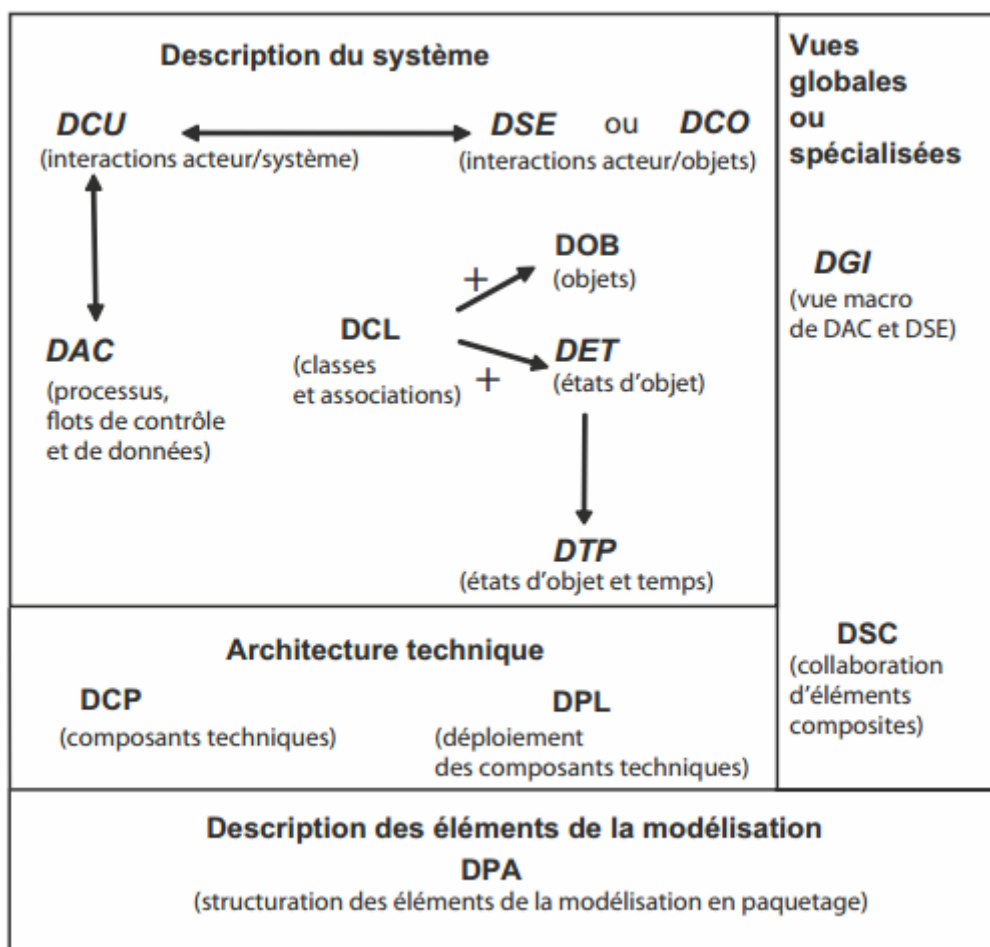


Figure 8: Schéma d'ensemble des treize diagrammes d'UML 2

Les abréviations de la figure ci-dessus sont définies dans le tableau suivant :

Abréviation	Description
DAC	Diagramme d'activité
DCL	Diagramme de classe
DOB	Diagramme d'objet
DCP	Diagramme de composant
DCU	Diagramme des cas d'utilisation
DCO	Diagramme de communication
DET	Diagramme d'état-transition
DGI	Diagramme global d'interaction
DPA	Diagramme de paquetage
DPL	Diagramme de déploiement
DSC	Diagramme de structure composite
DSE	Diagramme de séquence
DTP	Diagramme de temps

CHAPITRE II : LES DIAGRAMMES STRUCTURELS OU STATIQUES

I- Diagramme de Classe et diagramme d'objet

Le diagramme de classe constitue l'un des pivots essentiels de la modélisation avec UML.

En effet, ce diagramme permet de donner la représentation statique du système à développer. Cette représentation est centrée sur les concepts de classe et d'association.

Chaque classe se décrit par les données et les traitements dont elle est responsable pour elle-même et vis-à-vis des autres classes.

Les traitements sont matérialisés par des opérations. Le détail des traitements n'est pas représenté directement dans le diagramme de classe.

La description du diagramme de classe est fondée sur :

- le concept d'objet,
- le concept de classe comprenant les attributs et les opérations,
- les différents types d'association entre classes.

1. Objet (rappel)

Un objet est un concept, une abstraction ou une chose qui a un sens dans le contexte du système à modéliser. Chaque objet a une identité et peut être distingué des autres.

Un objet est caractérisé par les valeurs de ses propriétés qui lui confèrent des états significatifs suivant les instants considérés.

Le formalisme de représentation d'un objet est donné après celui d'une classe.

2. Classe, attribut et opération (rappel)

Classe

Une classe décrit un groupe d'objets ayant les mêmes propriétés (attributs), un même comportement (opérations), et une sémantique commune (domaine de définition).

Un objet est une instance d'une classe. La classe représente l'abstraction de ses objets.

Au niveau de l'implémentation, c'est-à-dire au cours de l'exécution d'un programme, l'identificateur d'un objet correspond une adresse mémoire.

Formalisme général et exemple

Une classe se représente à l'aide d'un rectangle comportant plusieurs compartiments.

Les trois compartiments de base sont :

- la désignation de la classe,
- la description des attributs,
- la description des opérations.

Deux autres compartiments peuvent être aussi indiqués :

- la description des responsabilités de la classe,
- la description des exceptions traitées par la classe.

NB : Il est possible de manipuler les classes en limitant le niveau de description à un nombre réduit de compartiments selon les objectifs poursuivis par le modélisateur.

La figure 9 montre le formalisme général des compartiments d'une classe et des premiers exemples.

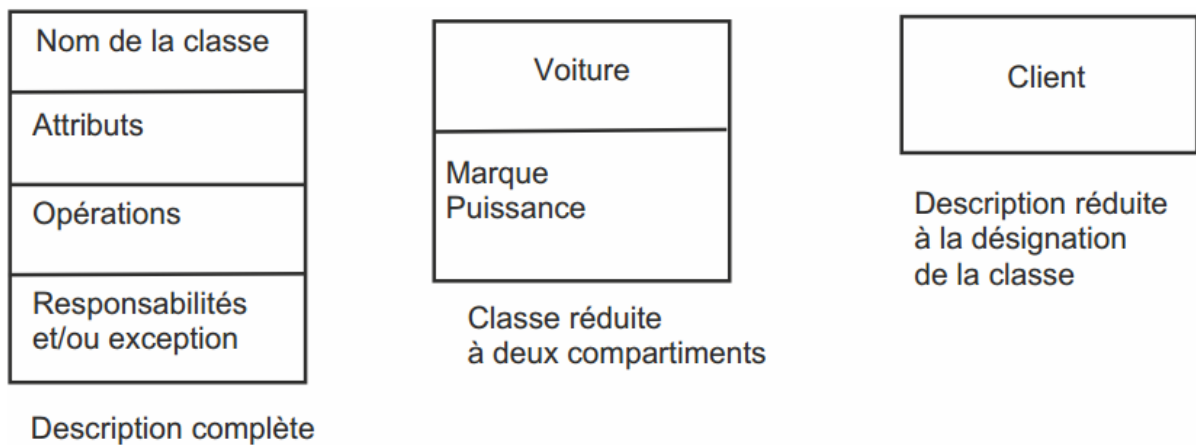


Figure 9:Exemples de représentations d'une classe

Attribut

Un attribut est une propriété élémentaire d'une classe. Pour chaque objet d'une classe, l'attribut prend une valeur.

Formalisme et exemple

La figure 10 montre le formalisme et un exemple de représentation des attributs de classe.

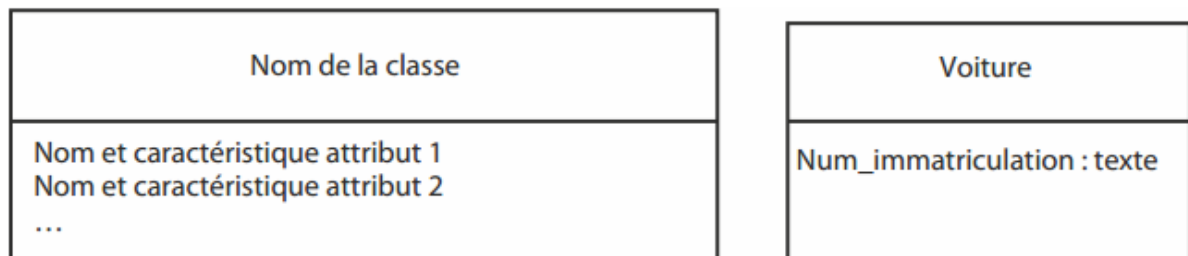


Figure 10: Formalisme d'attribut d'une classe

Caractéristiques

Le nom de la classe peut être qualifié par un « stéréotype ». La description complète des attributs d'une classe comporte un certain nombre de caractéristiques qui doivent respecter le formalisme suivant :

- Visibilité/Nom attribut : type [= valeur initiale {propriétés}]
 - Visibilité : se reporter aux explications données plus loin sur ce point.
 - Nom d'attribut : nom unique dans sa classe.

- Type : type primitif (entier, chaîne de caractères...) dépendant des types disponibles dans le langage d'implémentation ou type classe matérialisant un lien avec une autre classe.
- Valeur initiale : valeur facultative donnée à l'initialisation d'un objet de la classe.
- {propriétés} : valeurs marquées facultatives (ex. : « interdit » pour mise à jour interdite).

Un attribut peut avoir des valeurs multiples. Dans ce cas, cette caractéristique est indiquée après le nom de l'attribut (ex. : **prénom [3]** pour une personne qui peut avoir trois prénoms).

Un attribut dont la valeur peut être calculée à partir d'autres attributs de la classe est un attribut dérivé qui se note « /**nom de l'attribut dérivé** ».

Opération

Une opération est une fonction applicable aux objets d'une classe. Une opération permet de décrire le comportement d'un objet.

Une méthode est l'implémentation d'une opération.

Formalisme et exemple

Chaque opération est désignée soit seulement par son nom soit par son nom, sa liste de paramètres et son type de résultat. La signature d'une méthode correspond au nom de la méthode et la liste des paramètres en entrée. La figure 11 montre le formalisme et un exemple de représentation d'opérations de classe.

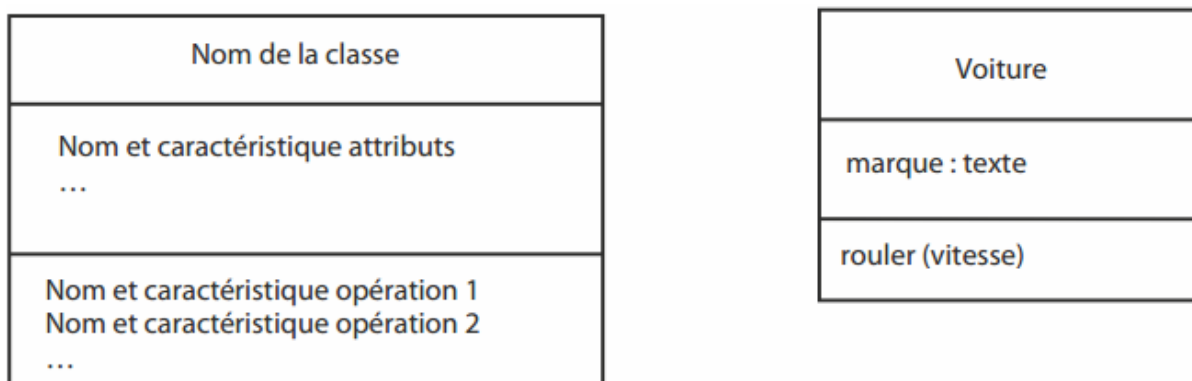


Figure 11 : Exemple d'opération d'une classe

Caractéristiques

La description complète des opérations d'une classe comporte un certain nombre de caractéristiques qui doivent respecter le formalisme suivant :

- Visibilité Nom d'opération (paramètres) : [[type résultat] {propriétés}]
 - **Visibilité** : se reporter aux explications données plus loin sur ce point.
 - **Nom d'opération** : utiliser un verbe représentant l'action à réaliser.
 - **Paramètres** : liste de paramètres (chaque paramètre peut être décrit, en plus de son nom, par son type et sa valeur par défaut). L'absence de paramètre est indiquée par ().
 - **Type résultat** : type de (s) valeur(s) retourné(s) dépendant des types disponibles dans le langage d'implémentation. Par défaut, une opération ne retourne pas de valeur, ceci est indiqué par exemple par le mot réservé « **void** » dans le langage C++ ou Java.
 - {propriétés} : valeurs facultatives applicables (ex. : {query} pour un comportement sans influence sur l'état du système).

Formalisme de représentation d'un objet

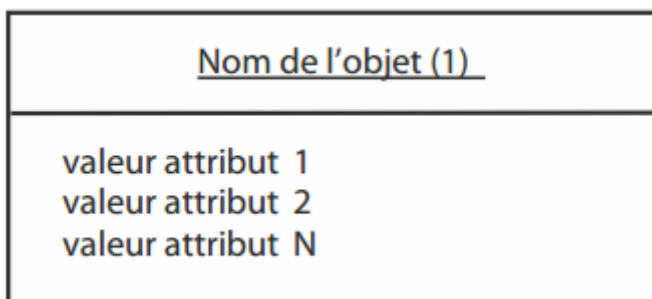


Figure 12 : Formalisme de représentation d'un objet

(1) Le nom d'un objet peut être désigné sous les formes suivantes :

- **nom de l'objet** (désignation directe et explicite d'un objet) ;
- **nom de l'objet : nom de la classe** (désignation incluant le nom de la classe) ;

Exercice :

Représenter la classe voiture par le formalisme UML, et quelques objets dérivés de cette classe.

Visibilité des attributs et opérations

Chaque attribut ou opération d'une classe peut être de type public, protégé, privé ou paquetage. Les symboles + (public), # (protégé), - (privé) et ~ (paquetage) sont indiqués devant chaque attribut ou opération pour signifier le type de visibilité autorisé pour les autres classes.

Les droits associés à chaque niveau de confidentialité sont :

- Public (+) : Attribut ou opération visible par tous.
- Protégé (#) : Attribut ou opération visible seulement à l'intérieur de la classe et pour toutes les sous-classes de la classe.
- Privé (-) : Attribut ou opération seulement visible à l'intérieur de la classe.
- Paquetage (~) : Attribut ou opération ou classe seulement visible à l'intérieur du paquetage où se trouve la classe

3. Association, multiplicité, navigabilité

Lien et association

Un lien est une connexion physique ou conceptuelle entre instances de classes donc entre objets. Une association décrit un groupe de liens ayant une même structure et une même sémantique. Un lien est une instance d'une association.

Chaque association peut être identifiée par son nom.

Une association entre classes représente les liens qui existent entre les instances de ces classes.

Formalisme et exemple

La figure 13 donne le formalisme de l'association. Le symbole (facultatif) indique le sens de lecture de l'association. Dans cette figure est donné aussi un exemple de représentation d'une association.

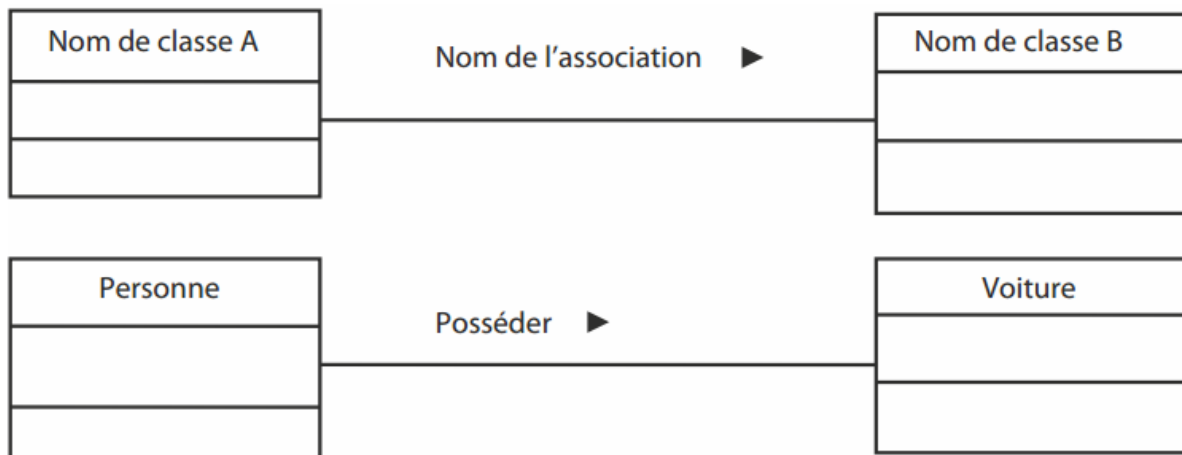


Figure 13: Formalisme et exemple d'association

Rôle d'association

Le rôle tenu par une classe vis-à-vis d'une association peut être précisé sur l'association.

Exemple :

La figure 14 donne un exemple de rôle d'association.



Figure 14: Exemple de rôle d'une association

Multiplicité

La multiplicité indique un domaine de valeurs pour préciser le nombre d'instance d'une classe vis-à-vis d'une autre classe pour une association donnée.

La multiplicité peut aussi être utilisée pour d'autres usages comme par exemple un attribut multivalué. Le domaine de valeurs est décrit selon plusieurs formes :

- Intervalle fermé – Exemple : 2, 3 ..15.
- Valeurs exactes – Exemple : 3, 5, 8.
- Valeur indéterminée notée * : 1..*.
 - Dans le cas où l'on utilise seulement *, cela traduit une multiplicité 0..*.
 - Dans le cas de multiplicité d'associations, il faut indiquer les valeurs minimale et maximale d'instances d'une classe vis-à-vis d'une instance d'une autre classe.

Formalisme et exemple

Nous donnons, à la figure 15, quelques exemples des principales multiplicités définies dans UML.

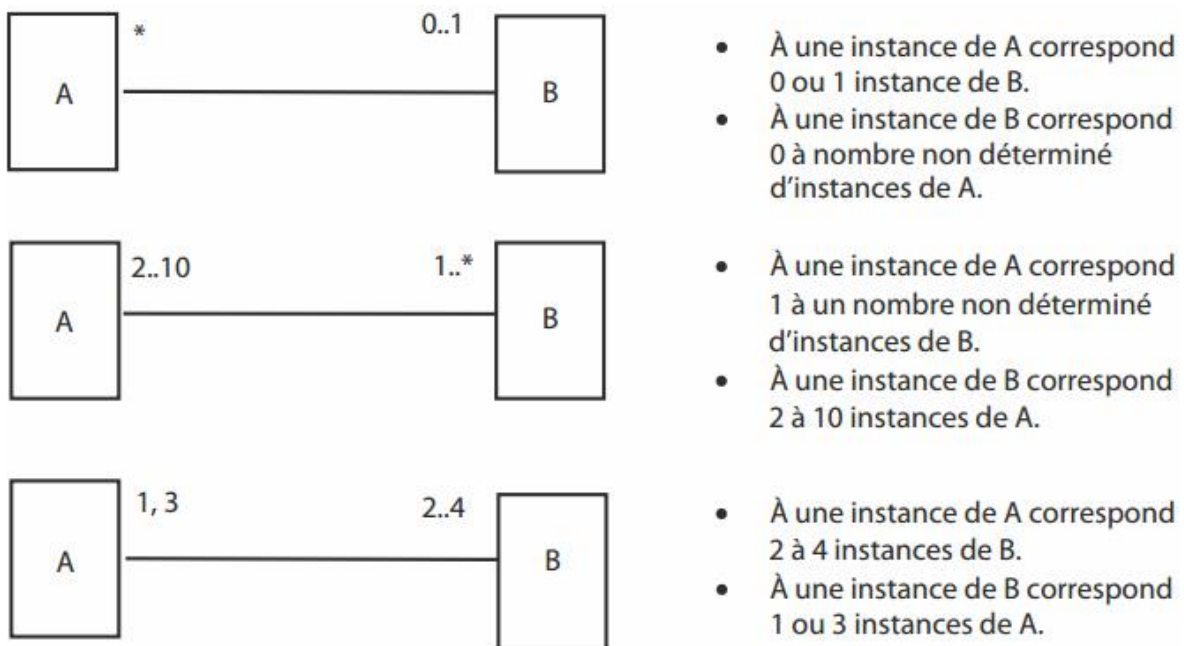


Figure 15 : Exemple de multiplicités

Navigabilité

La navigabilité indique si l'association fonctionne de manière unidirectionnelle ou bidirectionnelle, elle est matérialisée par une ou deux extrémités fléchées.

La non navigabilité se représente par un « X ».

Les situations possibles de navigabilité sont représentées à la figure 16.

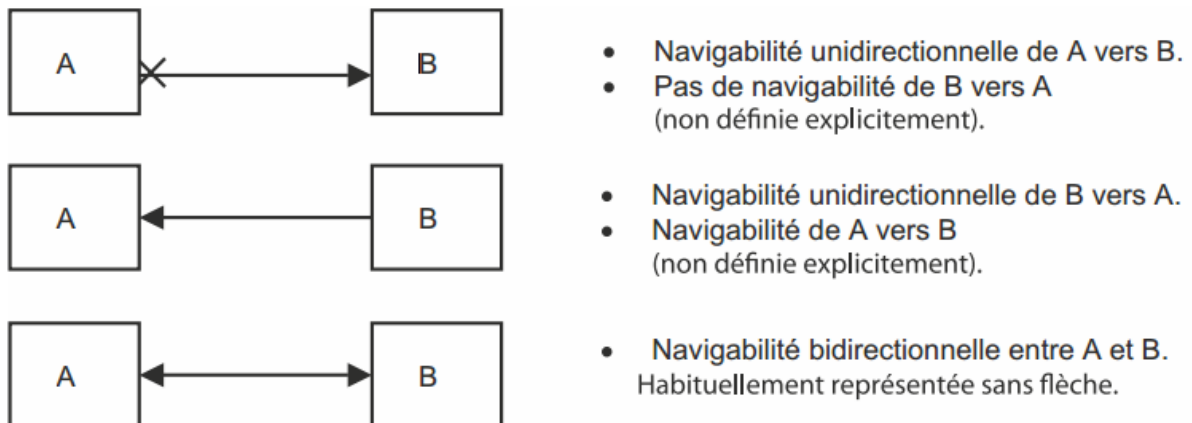


Figure 16 : Représentation de la navigabilité d'association

NB : Par défaut, on admet qu'une navigabilité non définie correspond à une navigabilité implicite.

Dans l'exemple donné à la figure 17, à une personne sont associées ses copies d'examen mais l'inverse n'est pas possible (retrouver directement l'auteur de la copie d'examen, notamment avant la correction de la copie).

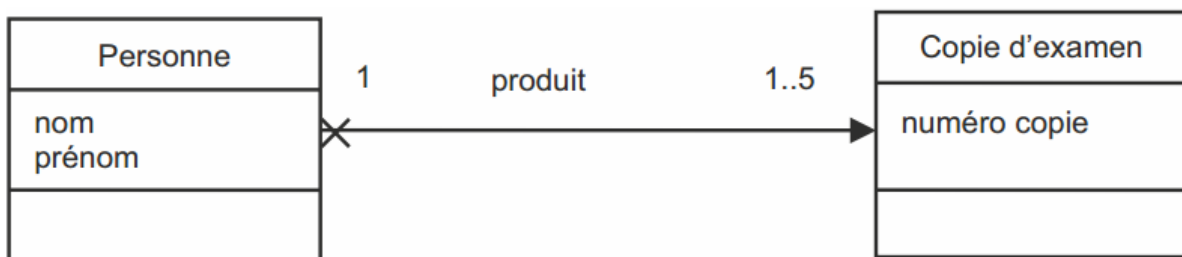


Figure 17 : Exemple de navigabilité d'une association

Association de dimension supérieure à 2 et classe-association

Une association de dimension supérieure à 2 se représente en utilisant un losange permettant de relier toutes les classes concernées.

Une classe-association permet de décrire soit des attributs soit des opérations propres à l'association.

Cette classe-association est elle-même reliée par un trait en pointillé au losange de connexion.

Une classe-association peut être reliée à d'autres classes d'un diagramme de classes.

Exemple

Un exemple d'une association de dimension 3 comprenant une classe-association « Affectation » est donné à la figure 18.

La classe-association Affectation permet de décrire les attributs propres à l'association de dimension 3 représentée.

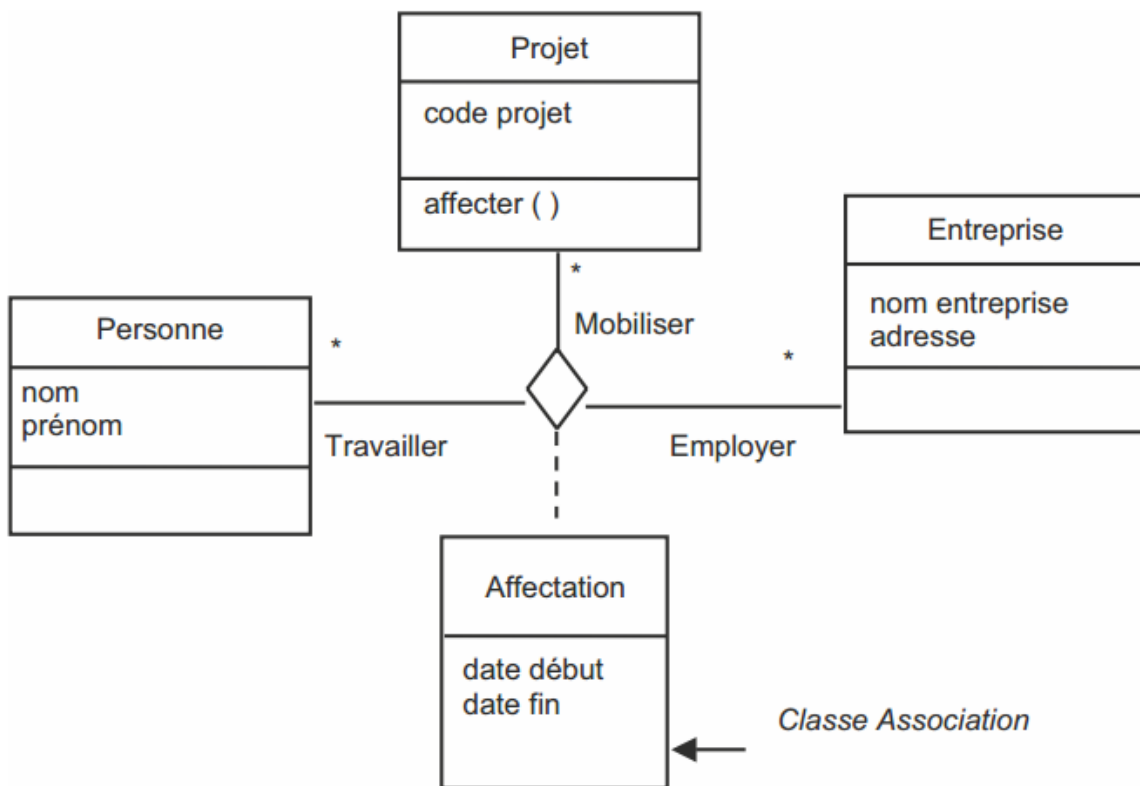


Figure 18: Exemple d'une association de dimension 3 et d'une Classe association

4. Agrégation et composition entre classes

Agrégation

L'agrégation est une association qui permet de représenter un lien de type « ensemble » comprenant des « éléments ».

Il s'agit d'une relation entre une classe représentant le niveau « ensemble » et 1 à n classes de niveau « éléments ».

L'agrégation représente un lien structurel entre une classe et une ou plusieurs autres classes. La figure 19 donne le formalisme général de l'agrégation.

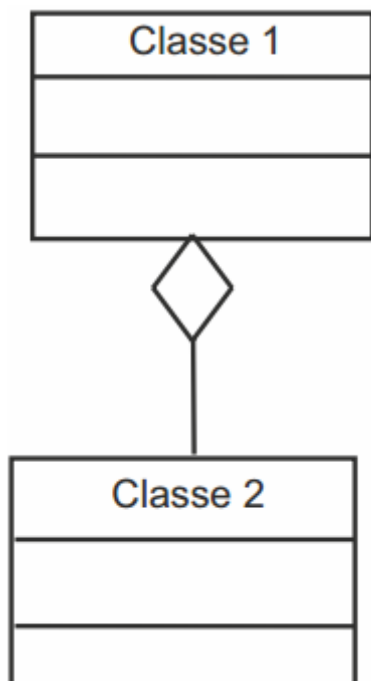


Figure 19 : Formalisme de l'agrégation

Exemple :

Dans cet exemple, nous avons modélisé le fait qu'un ordinateur comprend une UC, un clavier et un écran.

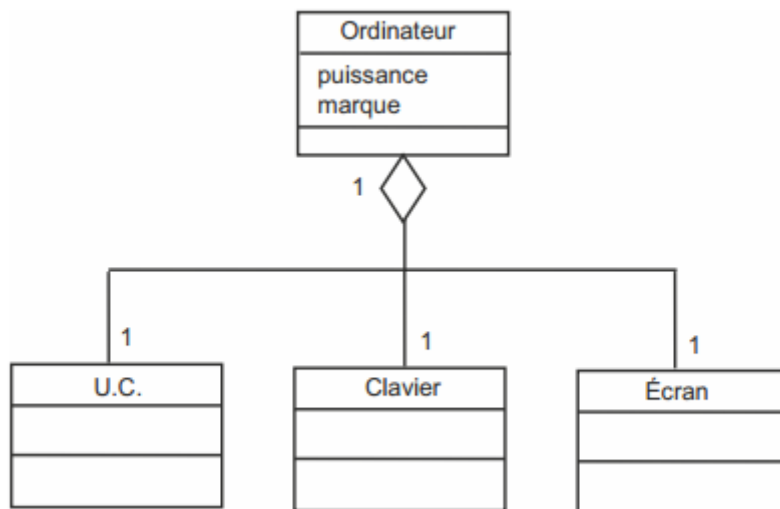


Figure 20 : Exemple d'agrégation

Composition

La composition est une relation d'agrégation dans laquelle il existe une contrainte de durée de vie entre la classe « composant » et la ou les classes « composé ».

Autrement dit la suppression de la classe « composé » implique la suppression de la ou des classes « composant ».

Formalisme et exemple

La figure 21 donne le formalisme général de la composition et la figure 22 montre un exemple de relation de composition.

Une seconde forme de présentation peut être aussi utilisée, elle est illustrée à la figure 23.

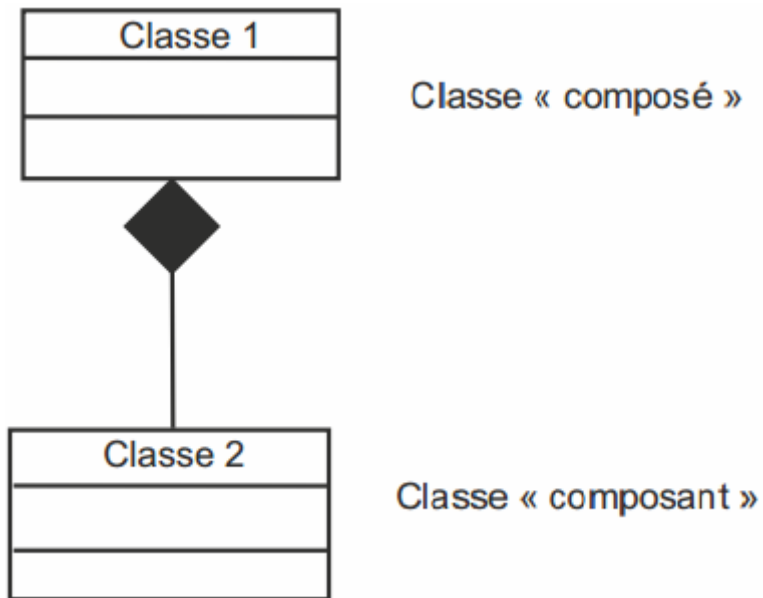


Figure 21 : Formalisme de composition

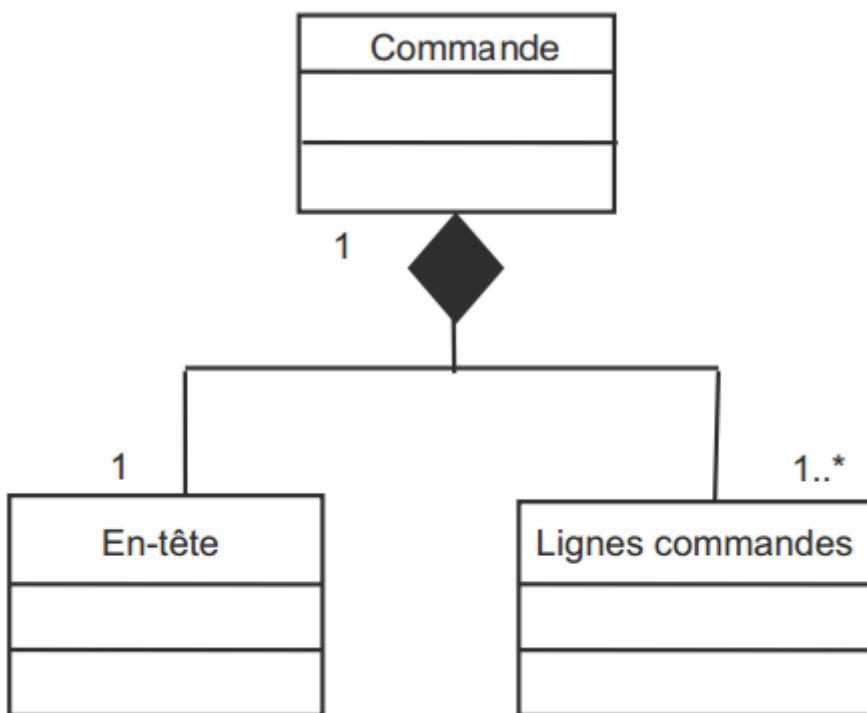


Figure 22 : Exemple d'une relation de composition

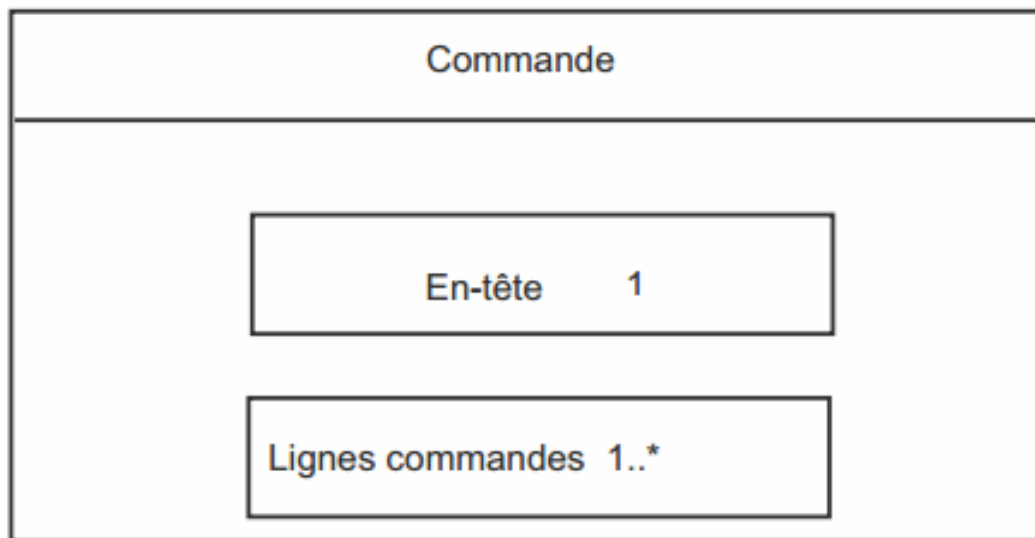


Figure 23 : Exemple d'une seconde forme de représentation

5. Dépendance et classe d'interface

Dépendance

La dépendance entre deux classes permet de représenter l'existence d'un lien sémantique.

Une classe B est en dépendance de la classe A si des éléments de la classe A sont nécessaires pour construire la classe B.

Formalisme et exemple

La relation de dépendance se représente par une flèche en pointillé (fig. 24) entre deux classes.



Figure 24 : Formalisme de lien de dépendance

Interface

Une classe d'interface permet de décrire la vue externe d'une classe.

La classe d'interface, identifiée par un nom, comporte la liste des opérations accessibles par les autres classes.

Le compartiment des attributs ne fait pas partie de la description d'une interface.

L'interface peut être aussi matérialisée plus globalement par un petit cercle associé à la classe source.

La classe utilisatrice de l'interface est reliée au symbole de l'interface par une flèche en pointillé.

La classe d'interface est une spécification et non une classe réelle.

Une classe d'interface peut s'assimiler à une classe abstraite.

Formalisme et exemple

La figure 25 donne le formalisme, sur un exemple, des deux types de représentation d'une interface.

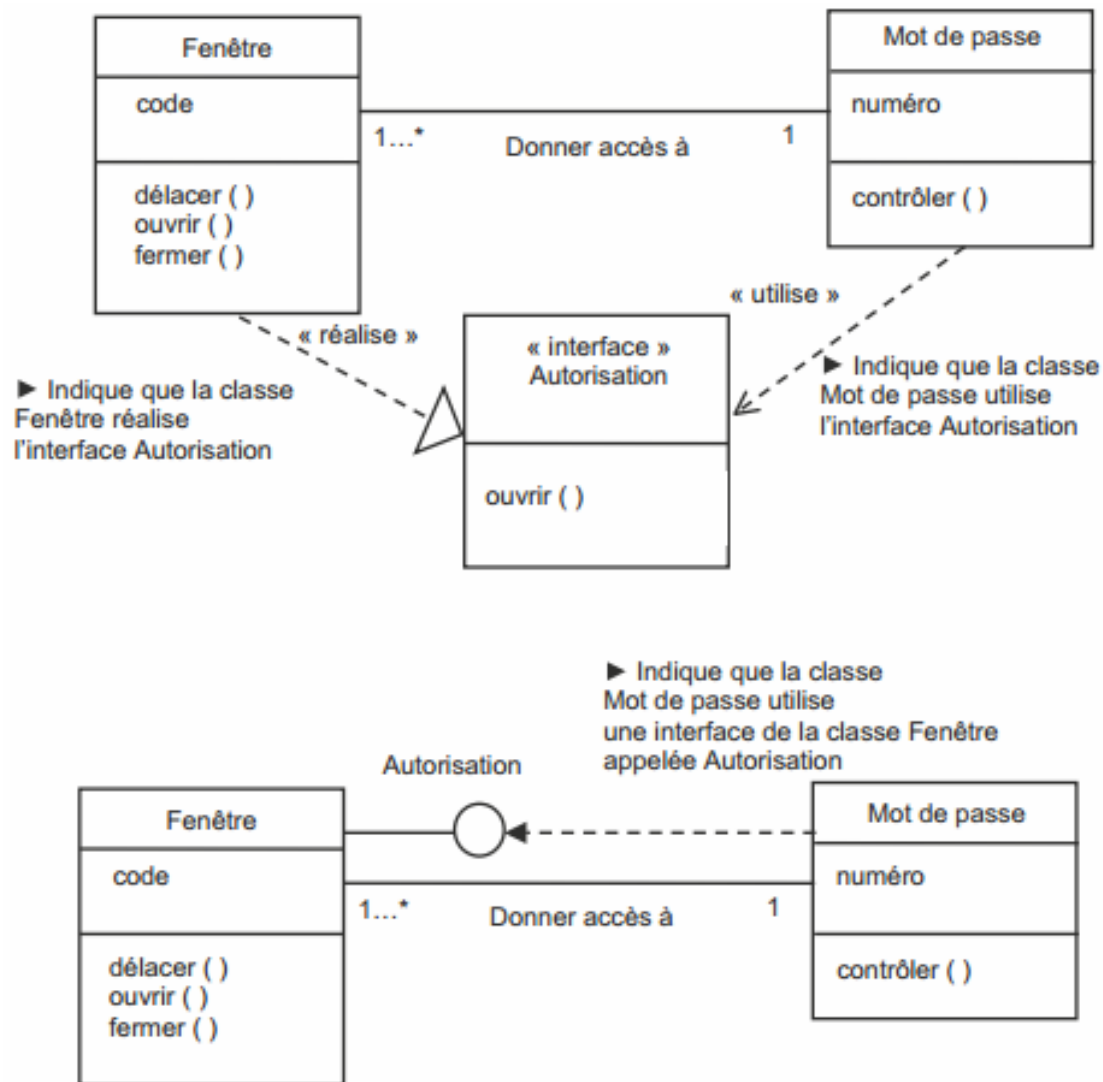


Figure 25 : Exemple de description d'une classe d'interface

6. Généralisation spécialisation

La généralisation/spécialisation et l'héritage simple (rappel)

La généralisation est la relation entre une classe et deux autres classes ou plus partageant un sous-ensemble commun d'attributs et/ou d'opérations.

La classe qui est affinée s'appelle super-classe, les classes affinées s'appellent sous-classes.

L'opération qui consiste à créer une super-classe à partir de classes s'appelle la généralisation.

Inversement la spécialisation consiste à créer des sous-classes à partir d'une classe.

Formalisme et exemple

La figure 26 montre le formalisme de la généralisation-spécialisation sous forme d'exemple général. Dans cet exemple :

- la sous-classe A1 hérite de A, c'est une spécialisation de A ;
- la sous-classe A2 hérite de A, c'est une spécialisation de A.

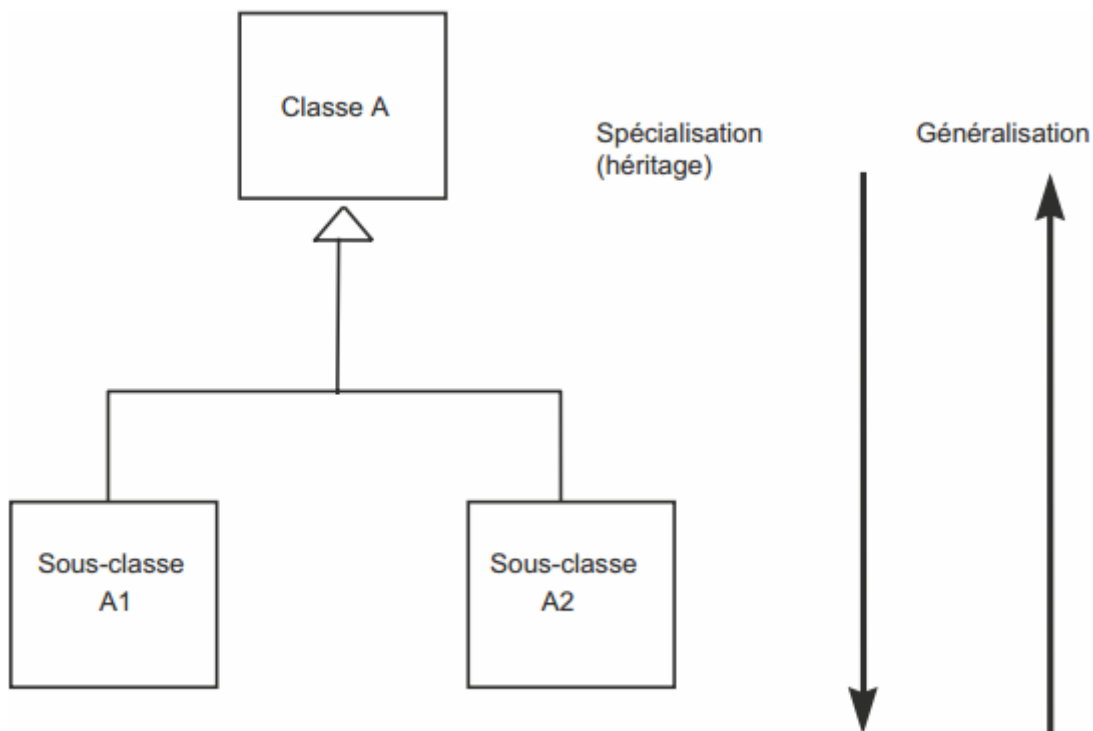


Figure 26: Formalisme de la relation de généralisation

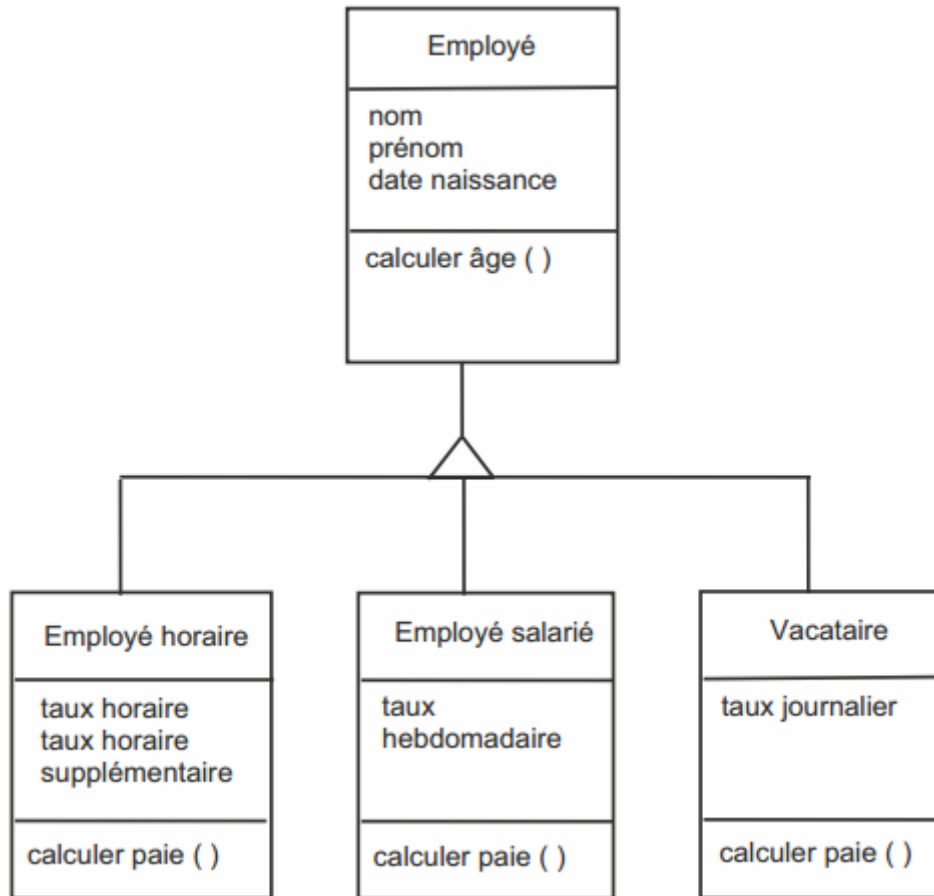


Figure 27: Exemple de généralisation

NB : Employé est une classe abstraite.

II- Diagramme d'objet

1. Objectif

Le diagramme d'objets représente les objets d'un système à un instant donné. Il permet de :

- Illustrer le modèle de classes (en montrant un exemple qui explique le modèle) ;
- Préciser certains aspects du système (en mettant en évidence des détails imperceptibles dans le diagramme de classes) ;
- Exprimer une exception (en modélisant des cas particuliers, des connaissances non généralisables. . .).

Le diagramme de classes modélise des règles et le diagramme d'objets modélise des faits.

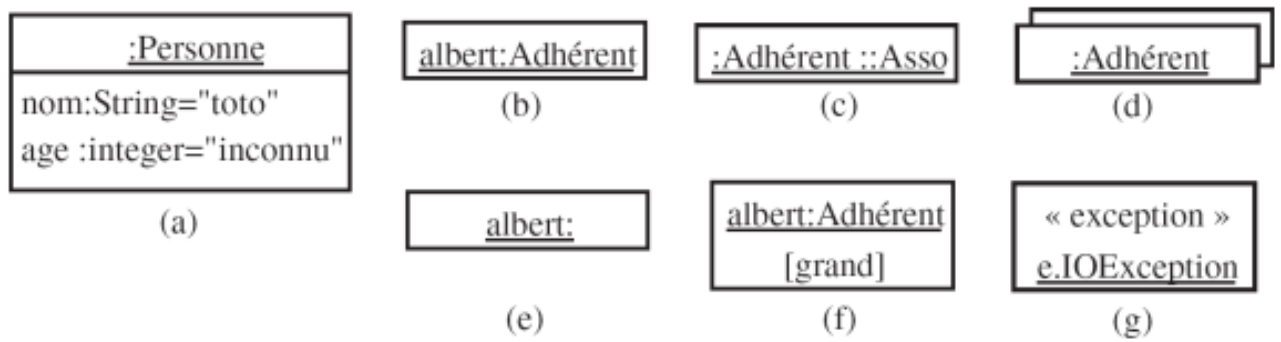
2. Représentation des objets

Comme les classes, on utilise des cadres compartimentés.

En revanche, les noms des objets sont soulignés et on peut rajouter son identifiant devant le nom de sa classe.

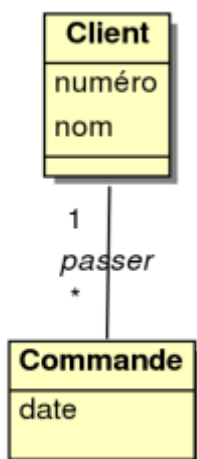
Les valeurs (a) ou l'état (f) d'un objet peuvent être spécifiées.

Les instances peuvent être anonymes (a,c,d), nommées (b,f), orphelines (e), multiples (d) ou stéréotypées (g).



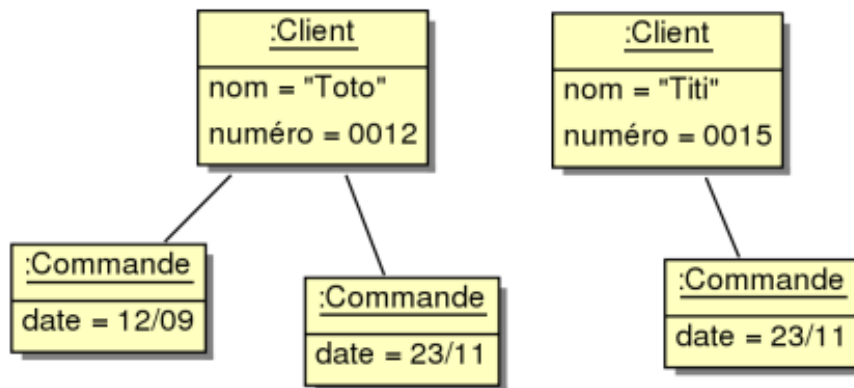
3. Diagramme de classes et diagramme d'objets

Le diagramme de classes *contraint* la structure et les liens entre les objets.

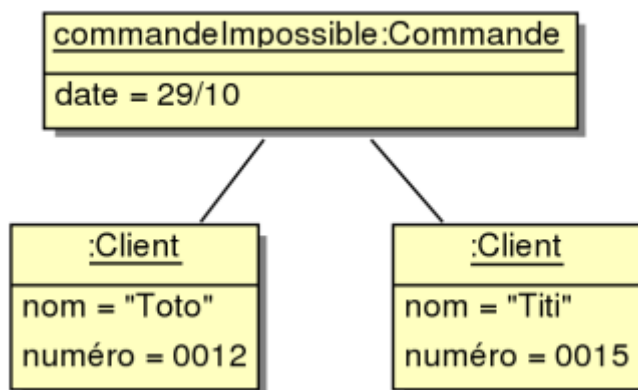


NB : Le diagramme d'objet doit est cohérent avec le diagramme de classes :

Cas de diagramme cohérent :



Cas de diagramme incohérent :



4. Liens

Un lien est une instance d'une association.

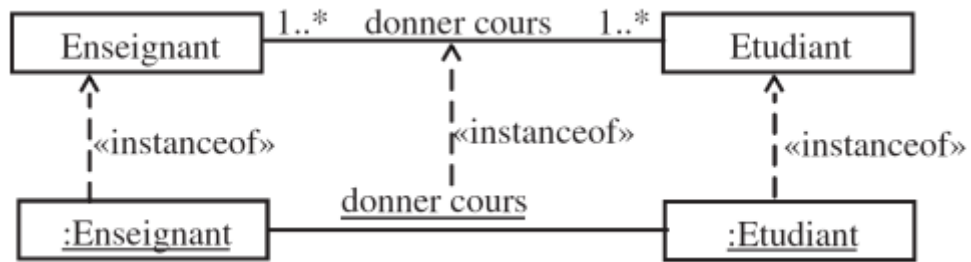
Un lien se représente comme une association mais s'il a un nom, il est souligné.

Attention : naturellement, on ne représente pas les multiplicités qui n'ont aucun sens au niveau des objets.

5. Relation de dépendance d'instanciation

La relation de dépendance d'instanciation (stéréotypée) décrit la relation entre un classeur et ses instances.

Elle relie, en particulier, les associations aux liens et les classes aux objets.



CHAPITRE III : LES DIAGRAMMES DE COMPORTEMENT

I- Diagramme de cas d'utilisation

1. Modélisation des besoins

Avant de développer un système, il faut savoir précisément à QUOI il devra servir, c'est-à-dire à quels besoins il devra répondre. Modéliser les besoins permet de :

- Faire l'inventaire des fonctionnalités attendues ;
- Organiser les besoins entre eux, de manière à faire apparaître des relations (réutilisations possibles, ...).

Avec UML, on modélise les besoins au moyen de diagrammes de cas d'utilisation.

Exemple de diagramme de cas d'utilisation

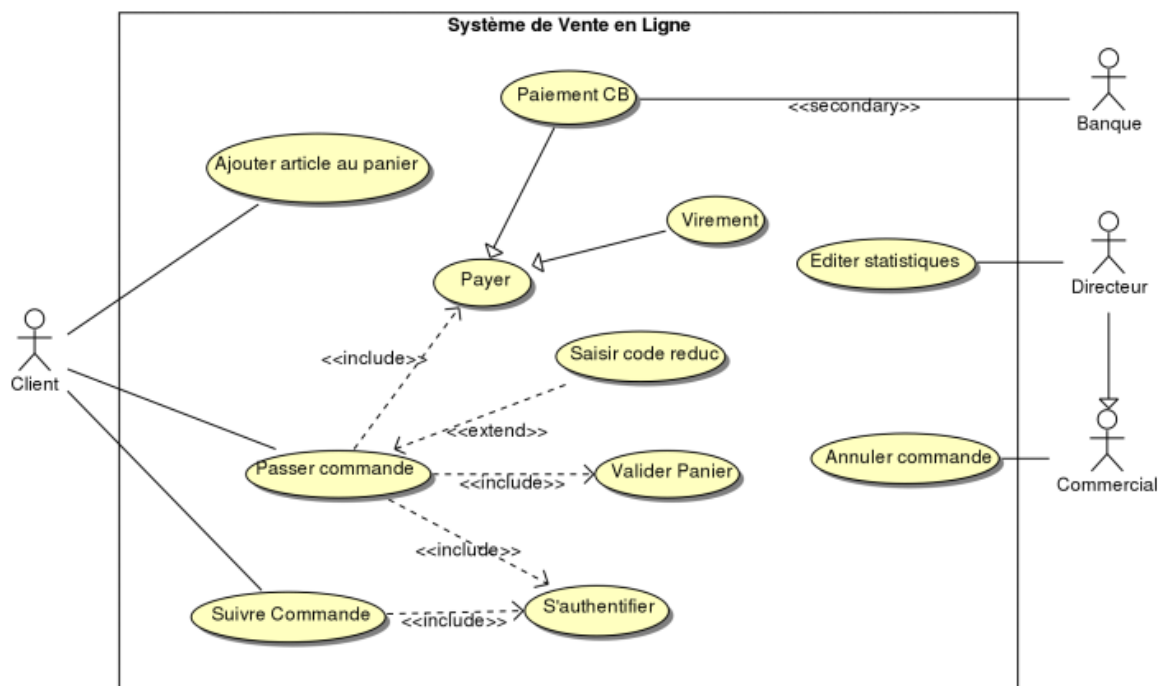


Figure 28: diagramme de cas d'utilisation d'un système de vente en Ligne

2. Cas d'utilisation

-Un cas d'utilisation est un service rendu à l'utilisateur, il implique des séries d'actions plus élémentaires.

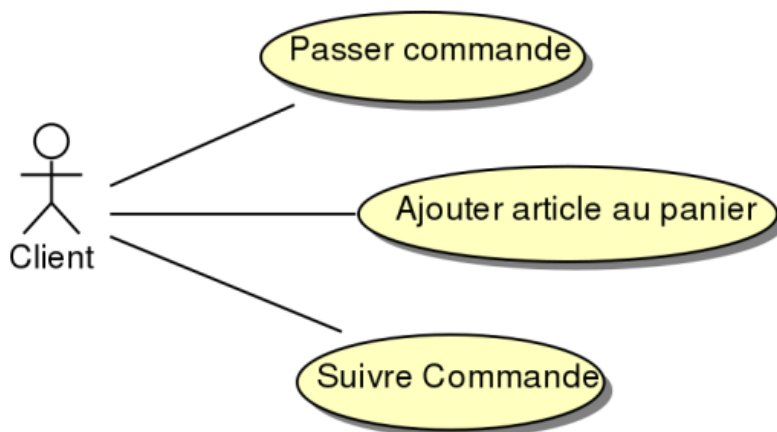


-Un acteur est une entité extérieure au système modélisé, et qui interagit directement avec lui.



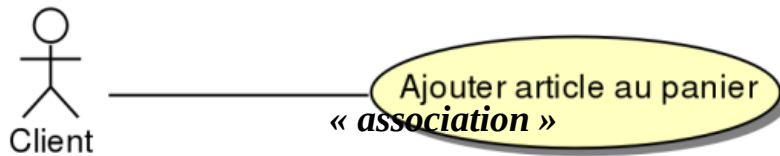
Un cas d'utilisation est l'expression d'un service réalisé de bout en bout, avec un déclenchement, un déroulement et une fin, pour l'acteur qui l'initie.

3. Acteurs et cas d'utilisation



Relations entre acteurs et cas d'utilisation

Les acteurs impliqués dans un cas d'utilisation lui sont liés par une association. Un acteur peut utiliser plusieurs fois le même cas d'utilisation.

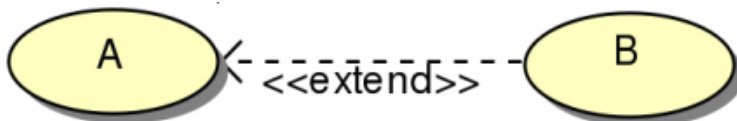


Relations entre cas d'utilisation

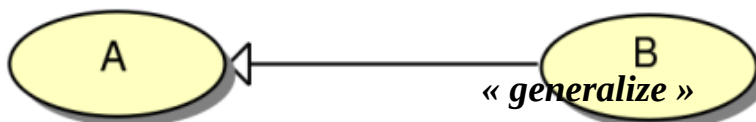
- **Inclusion** : le cas A inclut le cas B (B est une partie obligatoire de A)



- **Extension** : le cas B étend le cas A (A est une partie optionnelle de A)



- **Généralisation** : le cas A est une généralisation du cas du cas B (B est une sorte de A)



Dépendances d'inclusion et d'extension

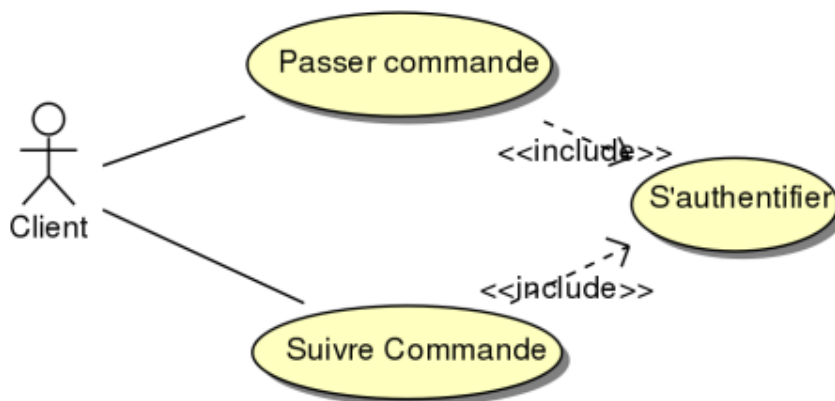
- Les inclusions et les extensions sont représentées par des dépendances.
 - Lorsqu'un cas B inclut un cas A, B dépend de A.
 - Lorsqu'un cas B étend un cas A, B dépend aussi de A.
 - On note toujours la dépendance par une flèche pointillée B ---> A qui se lit : «B dépend de A »
- Lorsqu'un élément A dépend d'un élément B, toute modification de B sera susceptible d'avoir un impact sur A.

- Les « **include** », « **extend** » et les « **generalize** » sont des stéréotypes (entre guillemets) des relations de dépendance.

Attention : le sens des flèches indique la dépendance, pas le sens de la relation d'inclusion.

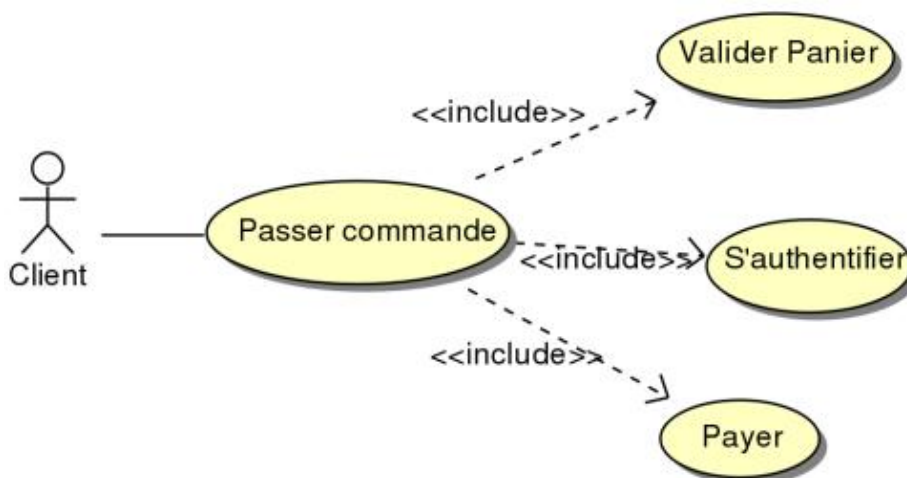
4. Réutilisabilité avec les inclusions et les extensions

Les relations entre cas permettent la **réutilisabilité** du cas « s'authentifier » : il sera inutile de développer plusieurs fois un module d'authentification.

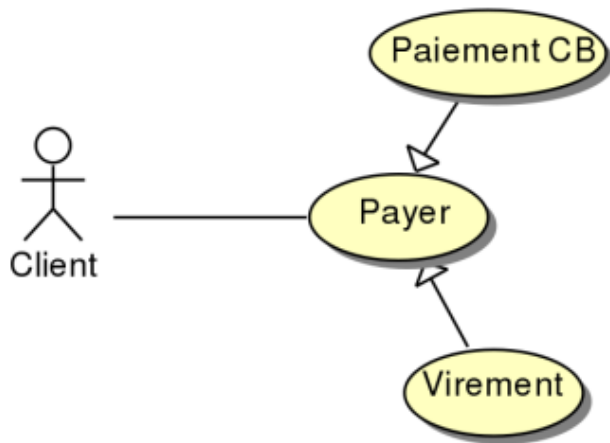


5. Décomposition grâce aux inclusions et aux extensions

Quand un cas est trop complexe (faisant intervenir un trop grand nombre d'actions), on peut procéder à sa décomposition en cas plus simples.



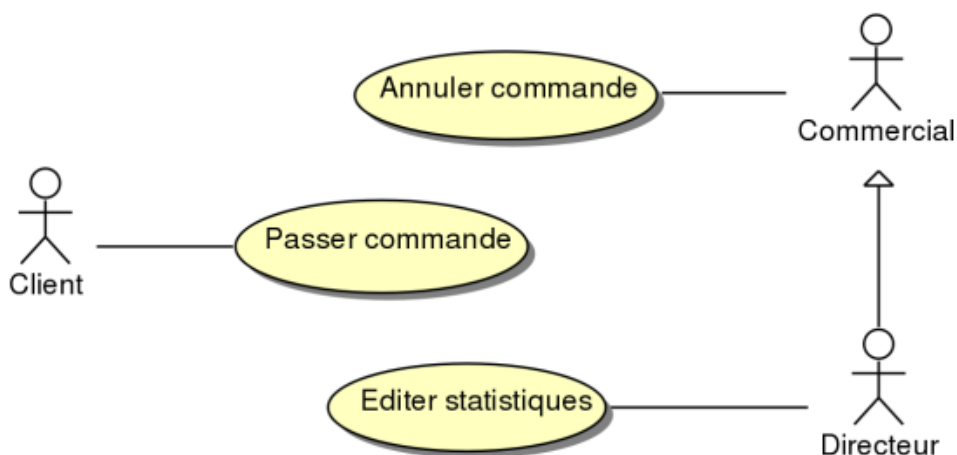
6. Généralisation



- Un virement est un cas particulier de paiement.
- Un virement est une sorte de paiement.
- La flèche pointe vers l'élément général.
- Cette relation de généralisation/spécialisation est présente dans la plupart des diagrammes UML et se traduit par le concept d'héritage dans les langages orientés objet.

7. Relations entre acteurs

Une seule relation possible : **la généralisation.**



8. Identification des acteurs

Les principaux acteurs sont les utilisateurs du système.

Attention :

- Un acteur correspond à un rôle, pas à une personne physique.
- Une même personne physique peut être représentée par plusieurs acteurs si elle a plusieurs rôles.
- Si plusieurs personnes jouent le même rôle vis-à-vis du système, elles seront représentées par un seul acteur.

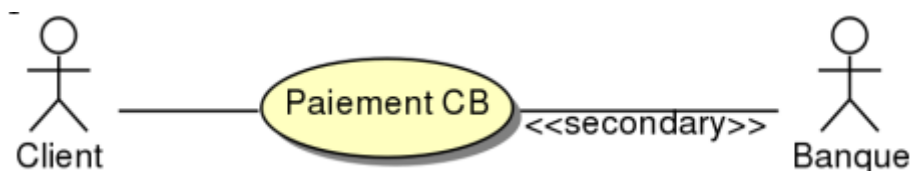
En plus des utilisateurs, les acteurs peuvent être :

- Des périphériques manipulés par le système (imprimantes...) ;
- Des logiciels déjà disponibles à intégrer dans le projet ;
- Des systèmes informatiques externes au système mais qui interagissent avec lui, etc.

Pour faciliter la recherche des acteurs, on se fonde sur les frontières du système.

9. Acteurs principaux et secondaires

L'acteur est dit principal pour un cas d'utilisation lorsque l'acteur est à l'initiative des échanges nécessaires pour réaliser le cas d'utilisation.



Les acteurs secondaires sont sollicités par le système alors que le plus souvent, les acteurs principaux ont l'initiative des interactions.

Le plus souvent, les acteurs secondaires sont d'autres systèmes informatiques avec lesquels le système développé est interconnecté.

10. Recenser les cas d'utilisation

Il n'y a pas une manière mécanique et totalement objective de repérer les cas d'utilisation. :

- Il faut se placer du point de vue de chaque acteur et déterminer comment il se sert du système, dans quels cas il l'utilise, et à quelles fonctionnalités il doit avoir accès.
- Il faut éviter les redondances et limiter le nombre de cas en se situant au bon niveau d'abstraction (par exemple, ne pas réduire un cas à une seule action).
- Il ne faut pas faire apparaître les détails des cas d'utilisation, mais il faut rester au niveau des grandes fonctions du système.

NB : Trouver le bon niveau de détail pour les cas d'utilisation est un problème difficile qui nécessite de l'expérience.

11.Description des cas d'utilisation

Le diagramme de cas d'utilisation décrit les grandes fonctions d'un système du point de vue des acteurs, mais n'expose pas de façon détaillée le dialogue entre les acteurs et les cas d'utilisation.

Un simple nom est tout à fait insuffisant pour décrire un cas d'utilisation.

Chaque cas d'utilisation doit être documenté pour qu'il n'y ait aucune ambiguïté concernant son déroulement et ce qu'il recouvre précisément.

Description textuelle

- Identification :
 - *Nom du cas* : Payer CB
 - *Objectif* : Détailler les étapes permettant à client de payer par carte bancaire
 - *Acteurs* : Client, Banque (secondaire)
 - *Date* : 24/10/2018
 - *Responsables* : Toto
 - *Version* : 1.0
- Séquencements :
 - Le cas d'utilisation commence lorsqu'un client demande le paiement par carte bancaire
 - *Préconditions* : Le client a validé sa commande

- *Enchaînement nominal*

1. Le client saisit les informations de sa carte bancaire
2. Le système vérifie que le numéro de CB est correct
3. Le système vérifie la carte auprès du système bancaire
4. Le système demande au système bancaire de débiter le client
5. Le système notifie le client du bon déroulement de la transaction

- *Enchaînements alternatifs*

1. En (2) : si le numéro est incorrect, le client est averti de l'erreur, et invité à recommencer
2. En (3) : si les informations sont erronées, elles sont redemandées au client

- *Post-conditions :*

- La commande est validée
- Le compte de l'entreprise est crédité

- Rubriques optionnelles

- *Contraintes non fonctionnelles :*

- Fiabilité : les accès doivent être sécurisés
 - Confidentialité : les informations concernant le client ne doivent pas être divulguées

- *Contraintes liées à l'interface homme-machine :*

- Toujours demander la validation des opérations bancaires

II- Diagramme de séquence

1. Objectif des diagrammes de séquence

Les diagrammes de cas d'utilisation modélisent à QUOI sert le système, en organisant les interactions possibles avec les acteurs.

Les diagrammes de classes permettent de spécifier la structure et les liens entre les objets dont le système est composé : ils spécifient QUI sera à l'œuvre dans le système pour réaliser les fonctionnalités décrites par les diagrammes de cas d'utilisation.

Les diagrammes de séquences permettent de décrire COMMENT les éléments du système interagissent entre eux et avec les acteurs.

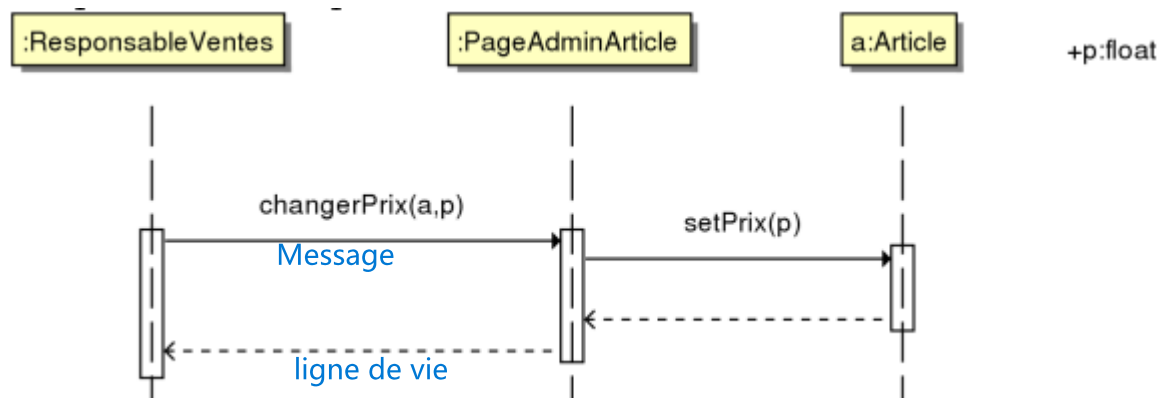
- Les objets au cœur d'un système interagissent en s'échangeant des messages.
- Les acteurs interagissent avec le système au moyen d'IHM (Interfaces Homme-Machine)

2. Exemple d'interaction

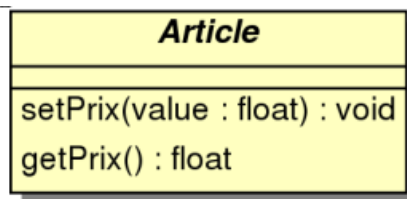
- Cas d'utilisation :



- Diagramme de séquences correspondant :



- Opérations nécessaires dans le diagramme de classes :

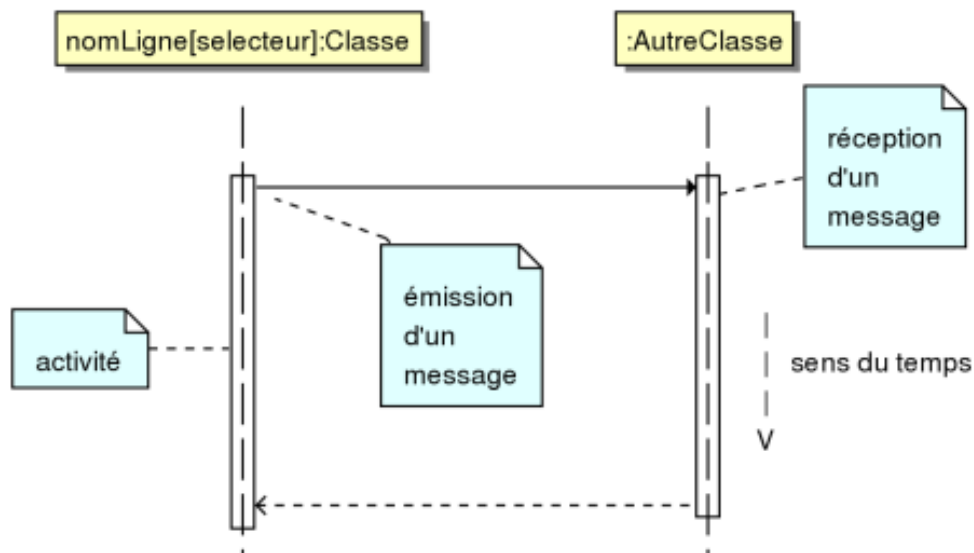


3. Ligne de vie

Une ligne de vie représente un participant à une interaction (objet ou acteur).

nomLigneDeVie [selecteur]: nomClasseOuActeur

Dans le cas d'une collection de participants, un sélecteur permet de choisir un objet parmi n (par exemple objets[2]).



4. Messages

Les principales informations contenues dans un diagramme de séquence sont les messages échangés entre les lignes de vie, présentés dans un ordre chronologique.

Un message définit une communication particulière entre des lignes de vie (objets ou acteurs).

Plusieurs types de messages existent, dont les plus courants :

- l'envoi d'un signal ;
- l'invocation d'une opération (appel de méthode) ;
- la création ou la destruction d'un objet.

La réception des messages provoque une période d'activité (rectangle vertical sur la ligne de vie) marquant le traitement du message (spécification d'exécution dans le cas d'un appel de méthode).

5. Principaux types de messages

Message synchrone

Un message **synchrone** bloque l'expéditeur jusqu'à la réponse du destinataire. Le flot de contrôle passe de l'émetteur au récepteur :

- Typiquement : appel de méthode
- Si un objet A invoque une méthode d'un objet B, A reste bloqué tant que B n'a pas terminé.



On peut associer aux messages d'appel de méthode un message de retour (en pointillés) marquant la reprise du contrôle par l'objet émetteur du message synchrone.

Message asynchrone

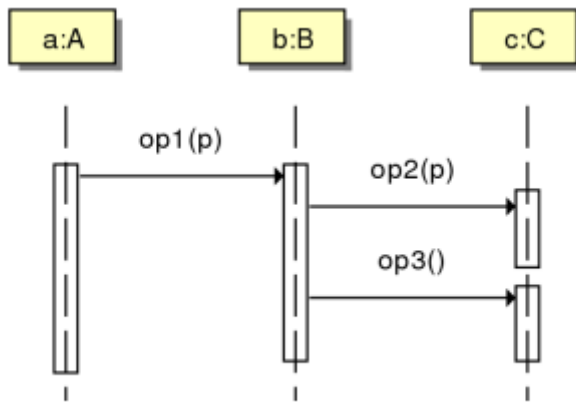
Un message **asynchrone** n'est pas bloquant pour l'expéditeur. Le message envoyé peut être pris en compte par le récepteur à tout moment ou ignoré.

Typiquement : envoi de signal (voir stéréotype de classe « signal »).

6. Correspondance messages / opérations

Les messages synchrones correspondent à des opérations dans le diagramme de classes.

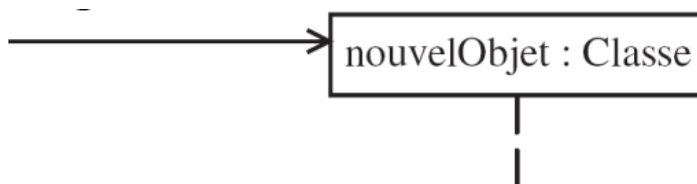
Envoyer un message et attendre la réponse pour poursuivre son activité revient à invoquer une méthode et attendre le retour pour poursuivre ses traitements.



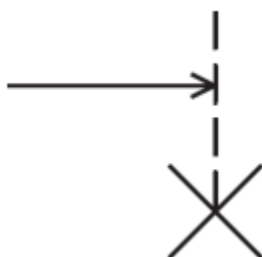
7. Création et destruction de lignes de vie

La **création** d'un objet est matérialisée par une flèche qui pointe sur le sommet d'une ligne de vie.

On peut aussi utiliser un message asynchrone ordinaire portant le nom « *create* ».



La **destruction** d'un objet est matérialisée par une croix qui marque la fin de la ligne de vie de l'objet.



8. Syntaxe des messages

La syntaxe des messages est :

nomSignalOuOperation (parametres)

La syntaxe des arguments est la suivante :

nomParametre = valeurParametre

Pour un argument modifiable :

nomParametre : valeurParametre

Exemples :

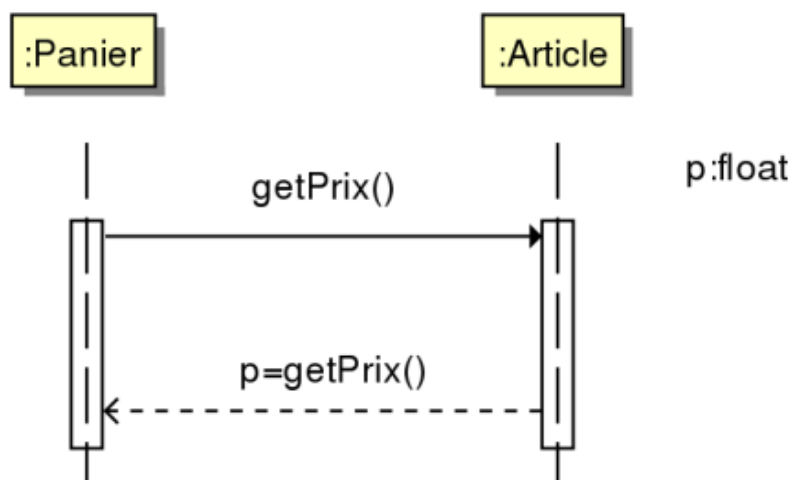
- appeler("Capitaine Hadock", 54214110)
- afficher(x,y)
- initialiser(x=100)
- f(x:12)

9. Messages de retour

Le récepteur d'un message synchrone rend la main à l'émetteur du message en lui envoyant un message de retour.

Les messages de retour sont optionnels : la fin de la période d'activité marque également la fin de l'exécution d'une méthode.

Ils sont utilisés pour spécifier le résultat de la méthode invoquée.



Le retour des messages asynchrones s'effectue par l'envoi de nouveaux messages asynchrones.

10. Syntaxe des messages de retour

La syntaxe des messages de retour est :

attributCible = nomOperation (params): valeurRetour

La syntaxe des paramètres est :

nomParam = valeurParam

Les messages de retour sont représentés en pointillés.

11.Fragment combiné

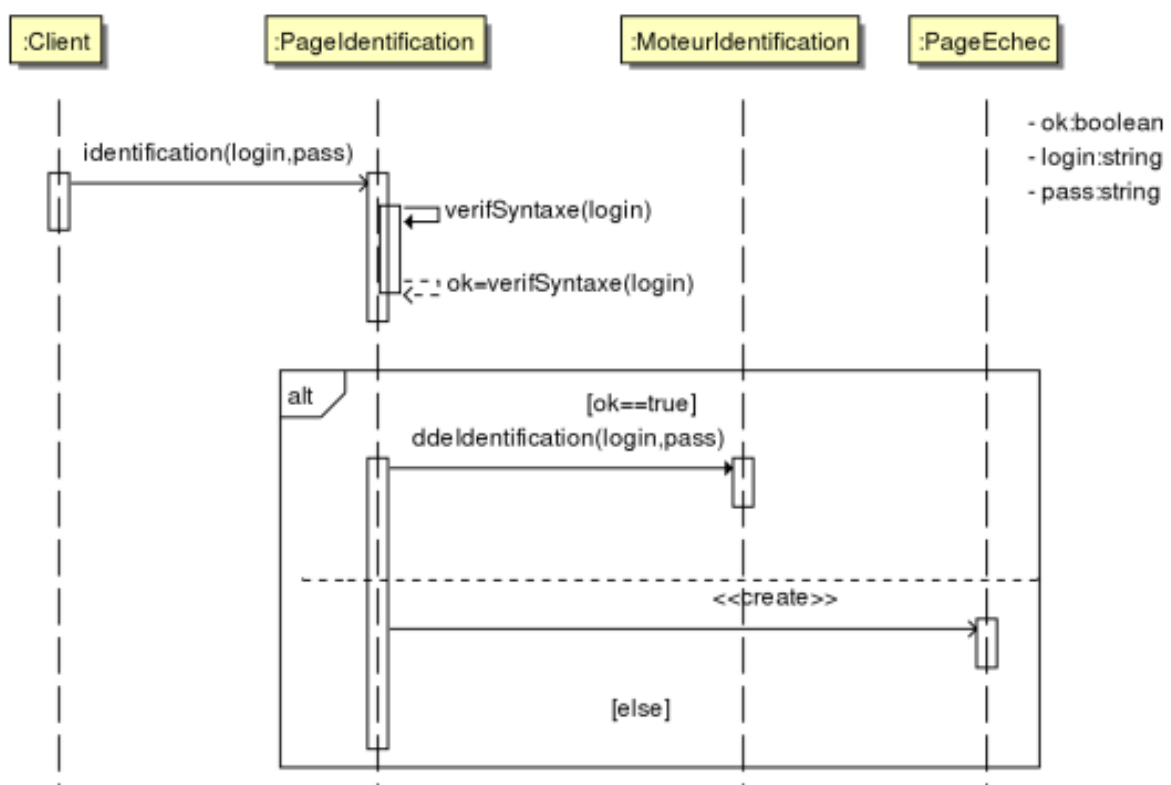
Un fragment combiné permet de décomposer une interaction complexe en fragments suffisamment simples pour être compris.

- Recombiner les fragments restitue la complexité.

Un fragment combiné se représente de la même façon qu'une interaction. Il est représenté un rectangle dont le coin supérieur gauche contient un pentagone :

- Dans le pentagone figure le type de la combinaison (appelé « **opérateur d'interaction** »)

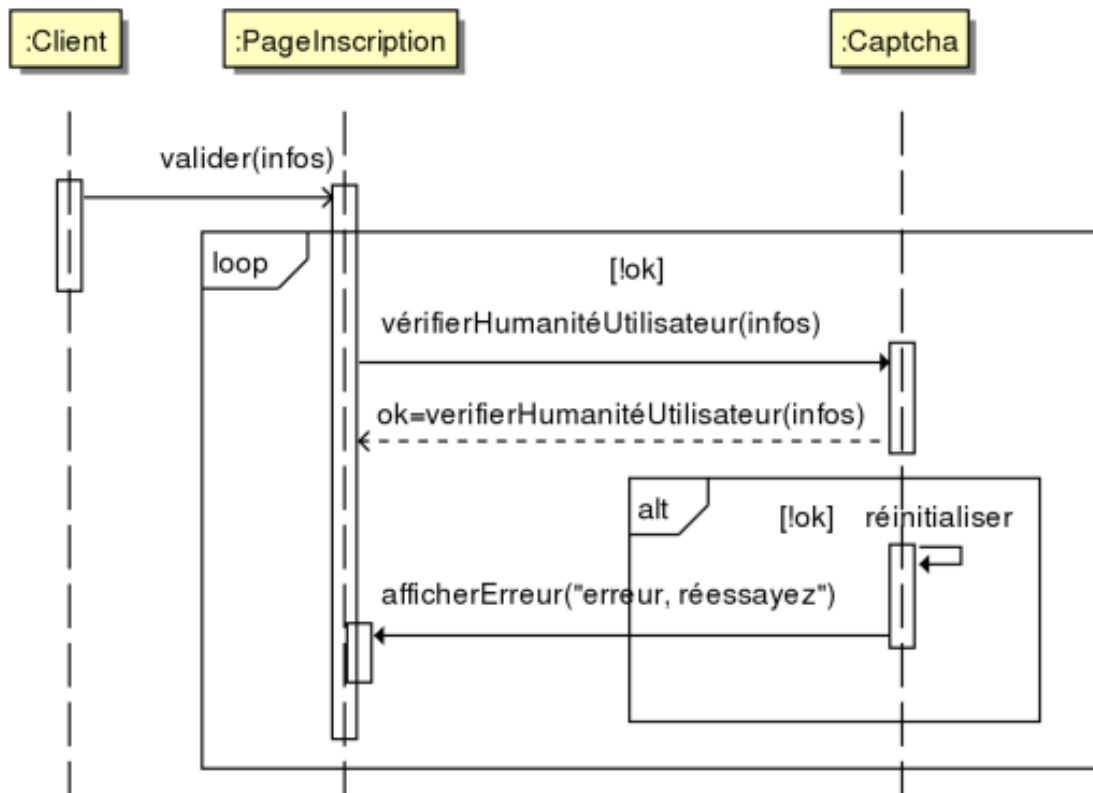
Exemple de fragment avec l'opérateur conditionnel



Type d'opérateurs d'interaction

- **Opérateurs de branchement** (choix et boucles) :
 - alternative, option, break et loop ;
- **Opérateurs contrôlant l'envoi en parallèle de messages** :
 - parallel et critical region ;
- **Opérateurs contrôlant l'envoi de messages** :
 - ignore, consider, assertion et negative ;
- **Opérateurs fixant l'ordre d'envoi des messages** :
 - weak sequencing et strict sequencing

Opérateur de boucle



Syntaxe de l'opérateur loop

Syntaxe d'une boucle : **loop (minNbIterations , maxNbIterations)**

La boucle est répétée au moins minNbIterations fois avant qu'une éventuelle condition booléenne ne soit testée (la condition est placée entre crochets dans le fragment)

Tant que la condition est vraie, la boucle continue, au plus maxNbIterations fois.

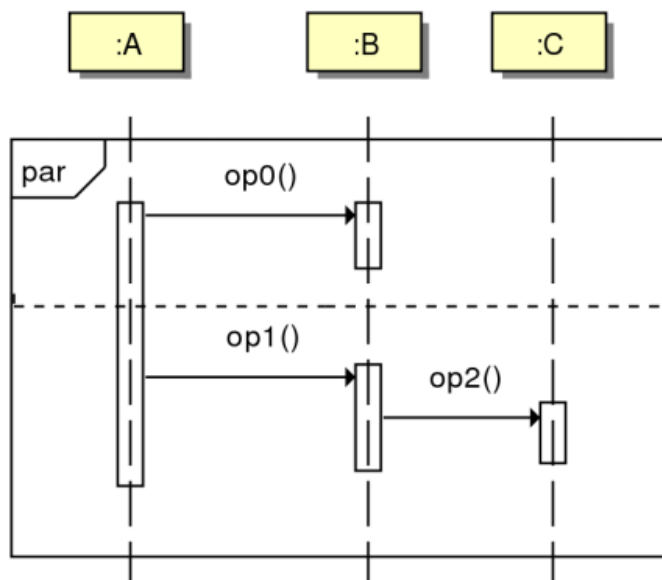
Notations :

- `loop(valeur)` est équivalent à `loop(valeur,valeur)`.
- `loop` est équivalent à `loop(0,*)`, où `*` signifie « illimité ».

Opérateur parallèle

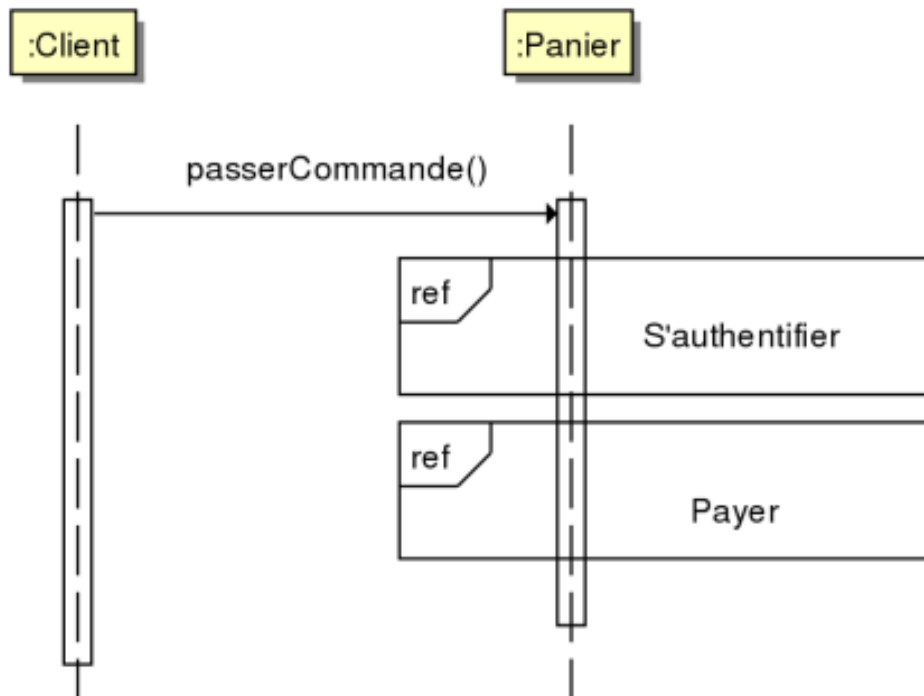
L'opérateur **par** permet d'envoyer des messages en parallèle.

Ce qui se passe de part et d'autre de la ligne pointillée est indépendant.

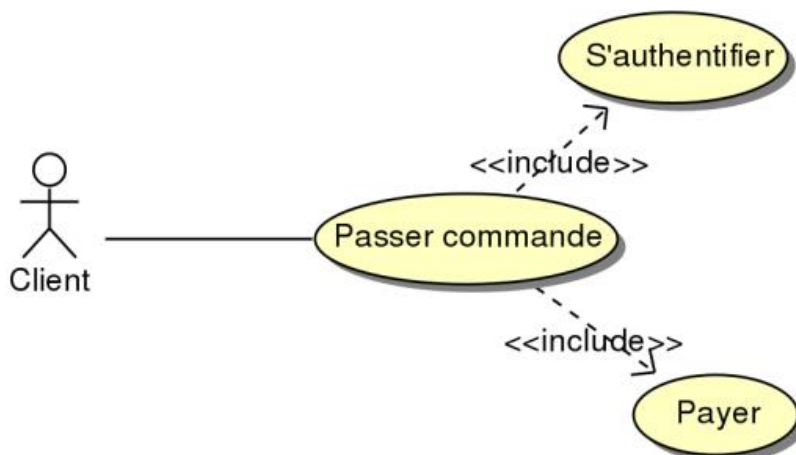


12. Réutilisation d'une interaction

Réutiliser une interaction consiste à placer un fragment portant la référence « ref » là où l'interaction est utile.



On spécifie le nom de l'interaction dans le fragment.



Les inclusions et les extensions sont des cas typiques d'utilisation de la réutilisation par référencement.

III- Diagramme d'état/transition

1. Concepts d'état, transition et évènement

L'état d'un objet est défini à un instant T par l'ensemble des valeurs de ses attributs.

Le passage d'un état à un autre s'appelle **la transition**. Cette transition peut être conditionnée.

Un évènement est un fait survenu qui déclenche une transition (signal, appel, temporel...)

Formalisme et exemple



2. Actions et activités

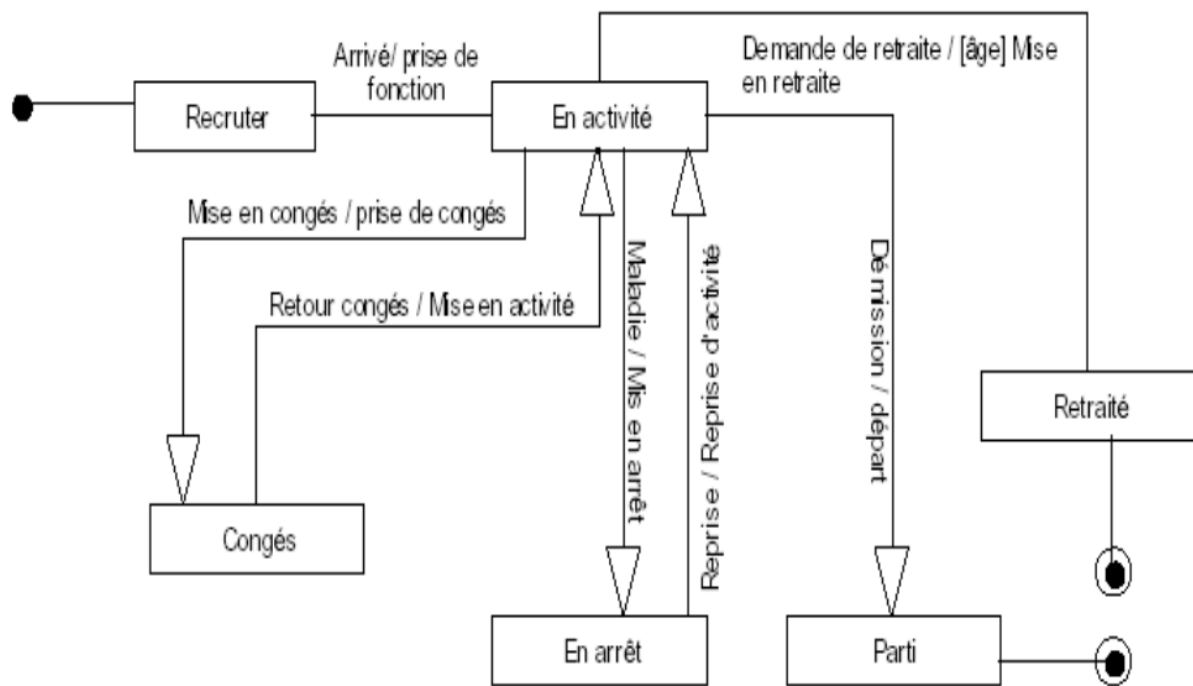
Une action est une opération instantanée qui ne peut être interrompue ; elle est associée à une transition.

Une activité est une opération d'une certaine durée qui ne peut être interrompue ; elle est associée à un état d'un objet.

Exemple 1 :

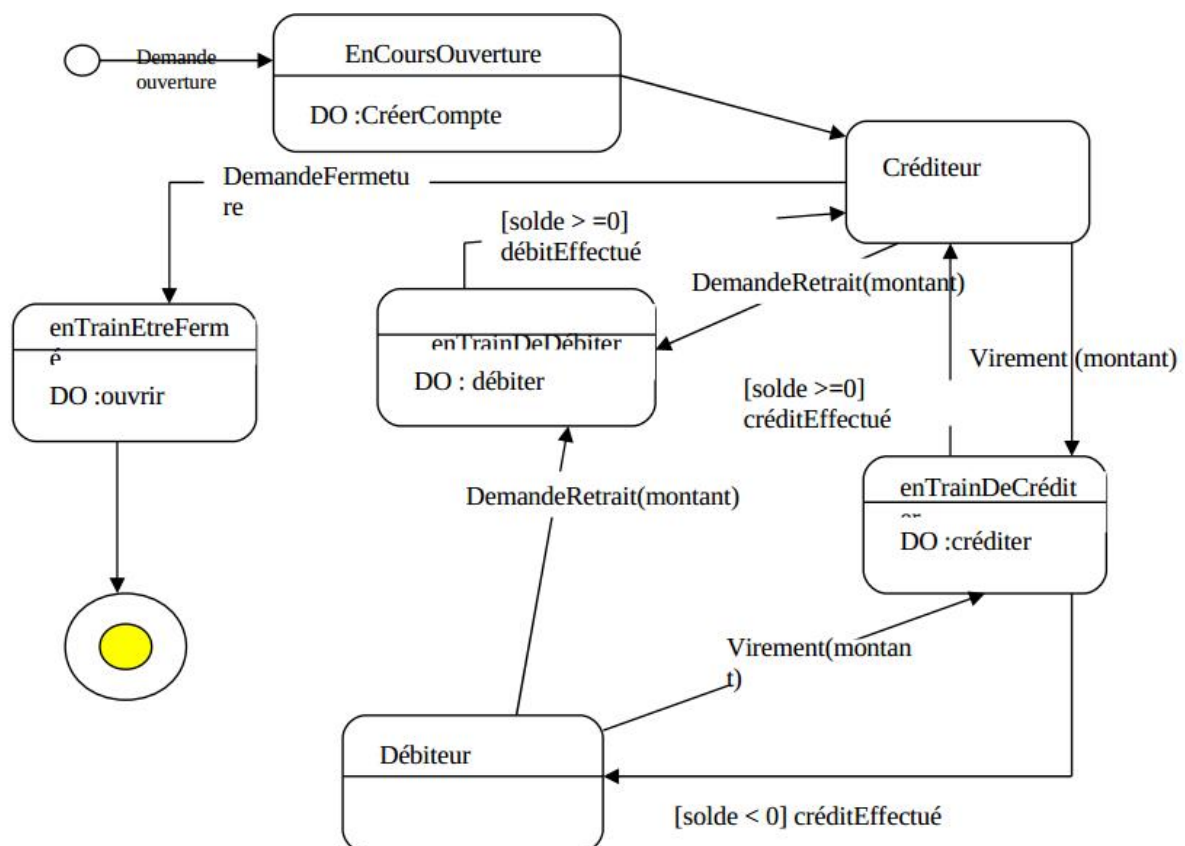
Représenter le diagramme d'état/transition d'un objet « *personnel* » suivant les évènements de gestion depuis le recrutement jusqu'à son départ. Après recrutement, une personne est considérée en activité dès sa prise de fonction. Les évènements à gérer sont :

- Le congé maladie
- La prise de congés
- La fin de carrière : Nous retiendrons deux situations : la démission et La retraite.



Exemple 2 :

Voici ci-dessous proposé un diagramme d'état d'un objet « compte ». Commentez ce diagramme et faites ressortir **les éventuelles anomalies**.



IV- Diagramme de collaboration

1. Utilité

Le diagramme de collaboration permet de mettre en évidence les interactions entre les différents objets du système étudié, ainsi que les messages qu'ils échangent entre eux.

A l'aide du diagramme de collaboration, nous illustrons donc l'interaction entre objets en créant des liens entre ces objets et en associant des messages à ces liens. Le nom d'un message doit évoquer l'intention de l'objet appelant lors de l'interaction avec l'objet associé.

2. Les messages

Les messages sont les seuls moyens de communication entre les objets. Ils sont donc positionnés sur le lien entre deux objets.

- Un **nom** est associé au message pour faciliter son identification.
- La **séquence** permet de préciser l'ordre d'émission des messages.
- Un message peut être complété par un ou plusieurs **arguments** : le message 5, ChoixCompteMontant a pour argument le compte et le montant choisis par le client (**voir schéma ci-dessous**).
- Certains messages peuvent solliciter un **résultat**. On peut représenter ce cas par deux messages : le premier fait la demande, le second apporte la réponse. Toutefois, lorsque le message en retour est immédiatement attendu, pour simplifier les diagrammes on peut les omettre.
- L'émission d'un message peut être soumise à une **garde** : Le contrôle doit être OK pour demander la liste des comptes d'un client.

Le diagramme de collaboration va nous permettre de compléter le modèle d'analyse commencé avec le diagramme de classe en ajoutant de nouvelles classes. Dans une première version du diagramme de classe nous nous sommes limités à l'étude des **classes entités**. Nous allons voir à l'aide du diagramme de collaboration qu'il est nécessaire d'avoir recours à d'autres types de classes pour gérer les interactions entre objets du système :

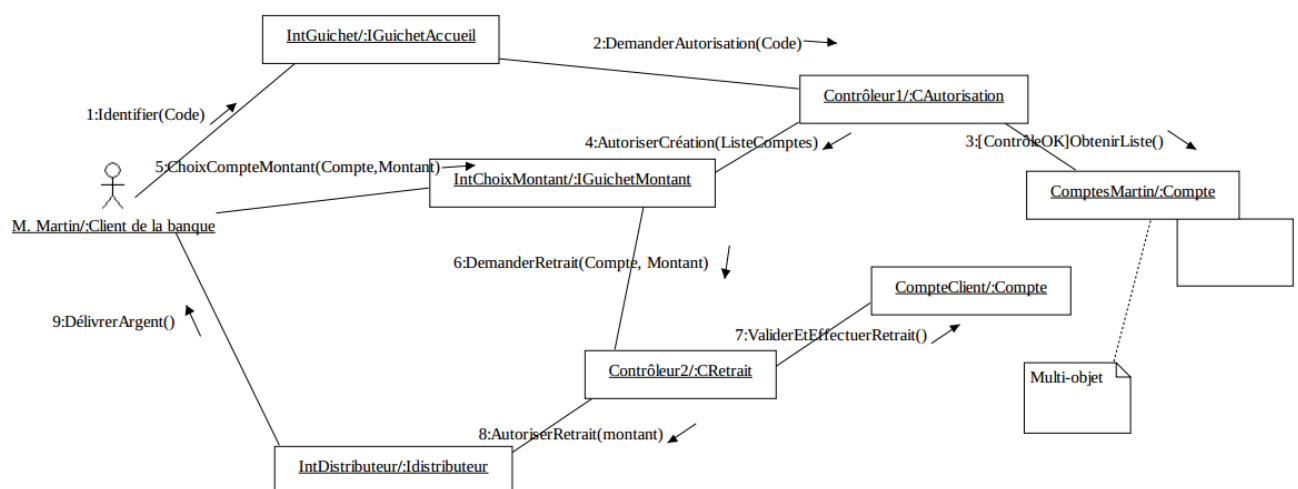
- **Les classes frontières (ou interfaces)** : servent à modéliser les interactions entre le système et ses acteurs ;

- **Les classes de contrôle** : ces classes encapsulent souvent le contrôle lié à un cas d'utilisation, ce qui implique qu'un objet de contrôle est créé au démarrage d'une instance de cas d'utilisation et prend fin à l'issue de ce scénario. Il y a néanmoins des exceptions : lorsqu'un objet de contrôle participe à la réalisation de plusieurs cas d'utilisation, lorsque plusieurs objets de contrôle (issus de différentes classes de contrôle) participent à la réalisation du cas d'utilisation, et enfin lorsqu'une réalisation de cas d'utilisation ne nécessite aucun objet de contrôle

En bref, un diagramme de collaboration permet de décrire les interactions entre objets intervenant dans la réalisation d'un scénario d'un cas d'utilisation.

Le diagramme de collaboration peut être complété par du texte décrivant de quelle façon les objets dialoguent pour effectuer le scénario du cas d'utilisation. C'est une description de scénario un peu plus précise.

Exemple : Utilisation d'un diagramme de collaboration pour décrire la réalisation du scénario « retrait autorisé » du cas d'utilisation « retirer de l'argent »



V- Diagramme d'activité

1. Définition

Un diagramme d'activités concerne le comportement d'un objet d'une classe déterminée.

Par exemple, la classe des commandes est notamment décrite par l'opération "contrôler bon de commande". Une méthode est l'implémentation d'une opération dans sa classe propriétaire. Elle décrit le processus permettant de réaliser l'opération.

Un diagramme d'activités sera utilisé pour la description d'opérations composées de suites d'actions élémentaires continues : le passage d'une action à sa suivante est fait automatiquement dès que l'action précédente est terminée. (Par opposition, si le passage d'une action à une autre se fait à cause de la survenance d'un événement externe, on utilisera plutôt une représentation par **Statecharts (diagramme d'état transition)**).

Un diagramme d'activités est donc utilisé pour des situations dans lesquelles un fil d'exécution continu est contrôlable de l'intérieur de l'objet.

Ce n'est pas le cas de la méthode *Commande :: contrôle* qui doit avoir recours à d'autres objets externes comme par exemple Client (pour obtenir le solde du compte) ou Produit (pour savoir s'il existe).

2. Le principe

C'est un découpage en étapes d'exécution qui utilise les concepts suivants :

Un état actif est une étape ou pas d'exécution de l'algorithme interne (avec un début, au moins une transition de sortie et sans événements extérieurs à cet état).

Une *transition* est le passage instantané et automatique à l'action suivante.

Une *décision* montre les conditions selon lesquelles, à la sortie d'une action, l'action suivante à exécuter (parmi plusieurs possibles) est déterminée.

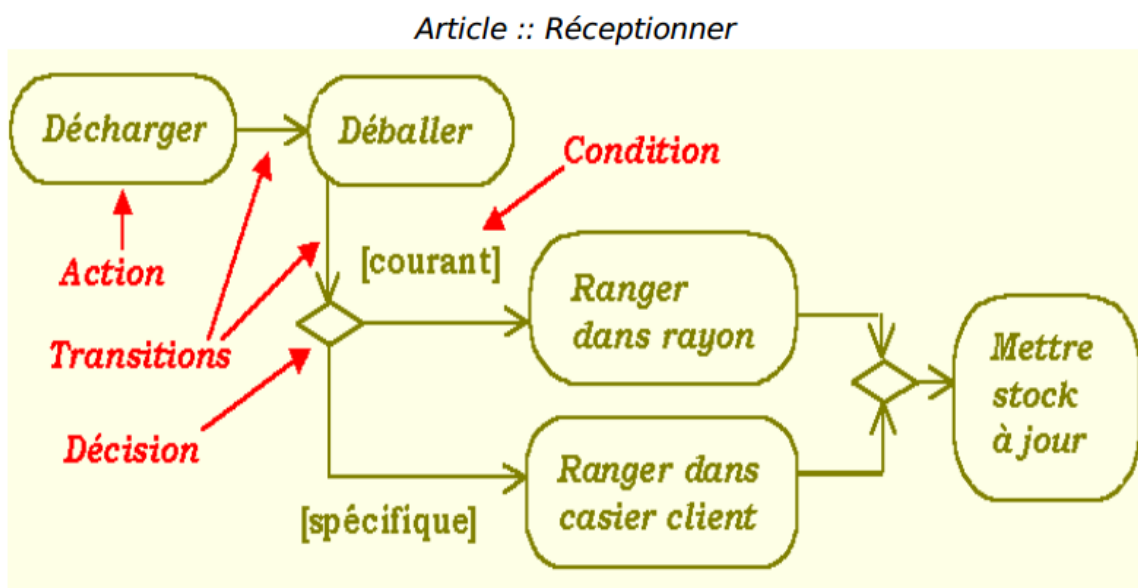
Un état actif peut invoquer un autre graphe d'activité complet. On dit qu'il entre alors dans un sous-état actif dont il ne peut sortir qu'à la terminaison du graphe invoqué. Un graphe d'activité donné peut être invoqué par plusieurs sous-états actifs.

Exemple :

Soit le cas d'une activité de Réception :

Mr Yao est chargé du magasin. Lors des livraisons, il trie les articles selon qu'il s'agit de choses courantes (elles sont rangées dans des rayons par familles d'articles) ou selon qu'il s'agit d'articles spécifiques à une affaire (rangés dans un casier spécial au nom du client).

Supposons qu'une classe **Article** a été défini comme possédant notamment une méthode **Réceptionner**.



Remarques :

Une transition est automatique : l'article est déballé dès qu'il est fini de décharger, il est rangé dès qu'il est fini de déballer, et le stock est mis à jour dès que l'article est rangé.

Les conditions (quelquefois appelées gardes) de la décision sont strictement alternatives, exhaustives et internes : *par exemple, il n'y a pas d'événement externe pour informer sur courant / spécifique, il n'y a pas d'autres cas possibles et ils sont exclusifs l'un de l'autre.*

On retrouve 3 figures habituelles des algorithmes : la séquence, l'alternative et le sous-programme (sous-états actifs).

L'exemple ne montre pas d'itération mais rien ne l'interdit.

Attention cependant : une itération ne peut concerner qu'une action ou un groupe d'actions à l'intérieur du diagramme d'activités. Si l'on voulait indiquer que l'activité elle-même se répète pour plusieurs articles livrés ensemble par exemple, ceci se ferait par application de la même activité (méthode) à plusieurs articles (objets, instances de classe) différents.

Des décisions successives peuvent s'enchaîner.

Les extensions UML pour le "Business Modeling" :

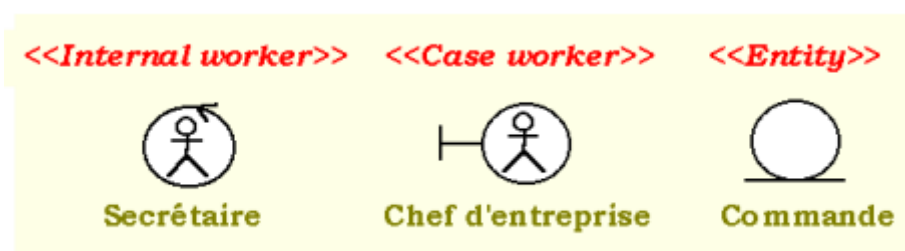
(NB : nous préférons garder l'expression anglaise des concepts plutôt que traduire, afin de marquer leur caractère de définitions au sein de la notation UML)

Worker : une abstraction d'une personne qui agit dans le système. Elle interagit avec d'autres personnes et manipule des entités. Elle se subdivise en :

- **Case worker** : une personne qui interagit avec des partenaires extérieurs au système : par exemple le Chef d'entreprise lorsqu'il négocie les devis avec les clients.
- **Internal worker** : une personne qui n'interagit avec d'autres qu'à l'intérieur du système. C'est le cas de notre secrétaire pour l'exemple ci-dessus.

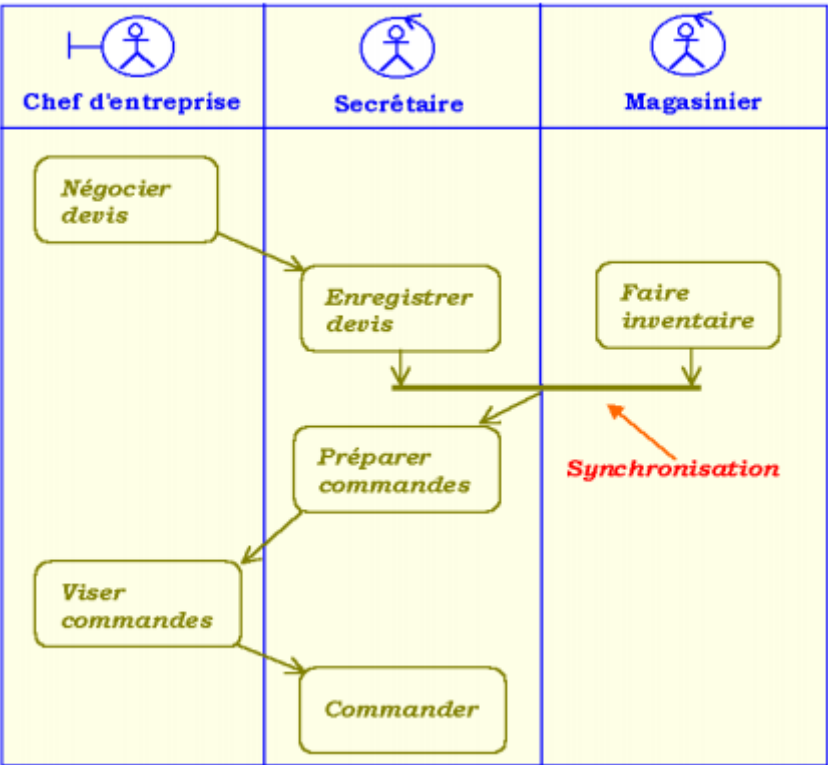
Entity : un objet passif qui n'a pas d'initiative propre d'interaction. L'entité sert de support au partage des activités entre plusieurs workers. Par exemple la commande est une entité créée par la secrétaire et visée par le chef d'entreprise.

Ces concepts peuvent être désignés par des icônes :



Les couloirs d'activités

Un couloir d'activité (swimlane) est une unité organisationnelle montrant la responsabilité des activités au sein d'une classe. Les interactions peuvent être montrées dans le diagramme par les échanges entre couloirs :



CHAPITRE IV : TRANSFORMATION RELATIONNELLE

1. Conception, génération de code

Généralement, on ne génère pas le code du LDD SQL directement à partir d'un diagramme de classes. On préfère transformer le diagramme de classes en un schéma représentant les données au travers de leurs éléments de modélisation relationnelle. Le code du LDD SQL est alors déduit du schéma relationnel. Un des intérêts est de pouvoir représenter la structure relationnelle des données sous une forme graphique donnant une vision générale des données, dégagée de toute contrainte syntaxique du langage du LDD SQL.

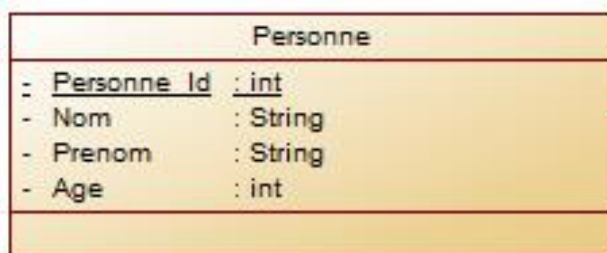
a. Transformation d'une classe dans le modèle relationnel

Principe général de transformation

Intuitivement, tel que le suggère la figure suivante, à chaque classe on peut faire correspondre une relation telle que :

- le nom de la relation reprend le nom de la classe,
- le nom des attributs de la relation sont issus des noms des attributs de la classe.

Exemple :



Personne(**Personne Id**, Nom, Prenom, Age)

Personne1 : Personne(1, 'Toto', 'Salif', 25)

Personne2 : Personne(1, 'Tata', 'Awa', 42)

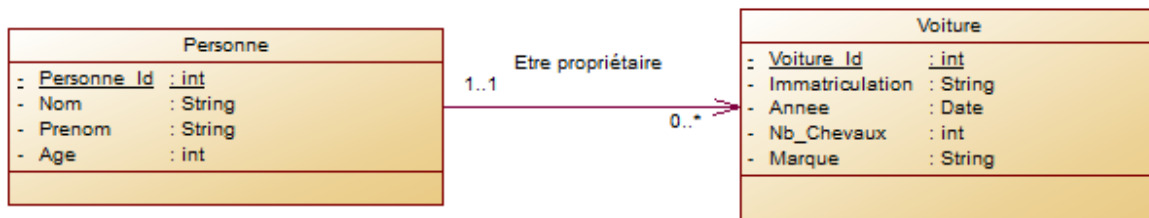
Code SQL> create table Personne(**Personne Id** int primary key, Nom varchar(30) , Prenom varchar(60));

Cette correspondance n'est directement possible que si les attributs de la classe sont atomiques (les types des attributs sont des types de base : Integer, String, ..., qui devront être adaptés, selon le cas, en type SQL : varchar, number ou date).

b. Principe de transformation des associations et des liens en relationnel

Association par valeur de clé (primaire) : cas de multiplicité 1..1 / 0..*

En relationnel, c'est au travers des valeurs de clé (primaire) que l'on peut lier (associer) des tuples entre eux. Supposons le diagramme classique suivant :



Personne(**Personne_Id**, Nom, Prenom, Age)

Voiture(**Voiture_Id**, Immatriculation, Annee, Nb_Chevaux, Marque)

La transformation en relationnel d'un tel diagramme se fait, intuitivement, selon les étapes suivantes :

- à chaque classe correspond une relation :

```

P1 : Personne(...) --- V1 : Voiture(...)
P2 : Personne(...) --- V2 : Voiture(...)
P3 : Personne(...) --- V3 : Voiture(...)
  
```

- puisque chaque voiture a un propriétaire qui est une personne, il suffit dans la relation Voiture de rajouter une colonne permettant d'indiquer, pour chaque voiture, quel est son propriétaire ; ce propriétaire pourra être

représenté par son identifiant, c'est-à-dire sa valeur de clé (primaire) ; il suffit donc de donner comme nom à cette colonne le nom du rôle :

Personne(**Personne_Id**, Nom, Prenom, Age)

Voiture(**Voiture_Id**, Immatriculation, Annee, Nb_Chevaux, Marque, leProprietaire)

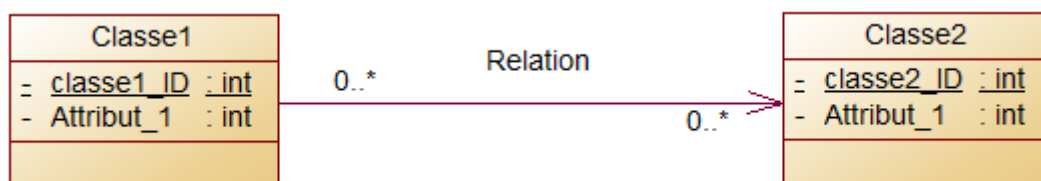
Puisqu'un propriétaire ne peut être qu'une personne qui existe, c'est-à-dire une personne dont le tuple est dans la relation Personne, toute valeur de la colonne Voiture(leProprietaire) doit être une valeur de clé primaire de la relation Personne, c'est-à-dire que toute valeur de la colonne(leProprietaire) doit se retrouver dans la colonne Personne(Personne_Id). On dit que l'attribut Voiture(leProprietaire) a pour domaine référentiel l'attribut Personne(Personne_Id). On écrit : **Voiture(leProprietaire) $\subseteq$ Personne(Personne_Id).**

Le code du LDD SQL, correspondant au schéma relationnel, est le suivant :

SQL> create table Personne(Personne_Id varchar(10) primary key,...);

SQL> create table Voiture(Voiture_Id varchar(10) primary key, ...
leProprietaire varchar(10) references Personne(Personne_Id));

Association par valeur de clé (primaire) : cas de multiplicité 0..* / 0..*



Classe1(**classe1_ID**,Attribut_1,...)

Classe2(**classe2_ID**,Attribut_2,...)

Relation(**Relation_ID**, cleClasse1,cleClasse2)

Génération du code du LDD SQL :

SQL> create table Classe1(classe1_ID int primary key, Attribut_1,...);

```
SQL> create table Classe2(classe2_ID int primary key, Attribut_1,...);
```

```
SQL> create table Relation (Relation_ID int primary key, cleClasse1 int  
references Classe1(classe1_ID), cleClasse2 int references Classe2(classe2_ID)) ;
```

Exemple: un étudiant peut s’inscrire dans plusieurs modules. Modéliser et donner la transformation relationnelle puis le cde SQL correspondant.

2. Algorithme de transformation d’une association en relationnel

1. Chaque classe du diagramme de classes devient une table en relationnel.
2. A chaque classe du diagramme de classe est associé un identifiant : celui-ci est soit un identifiant à générer « classe_id » soit une propriété de la classe. Cet identifiant devient la clé primaire de la table créée correspondante.
3. Toute relation dont une des cardinalités est de type (... ,1) provoque la génération d’une clé étrangère. Cela a pour conséquence la création d’une colonne de nom « le_rôle » dans la table, chaque valeur de cette colonne doit faire référence à la clé primaire de l’autre table issue de l’association.
4. Pour toutes les autres cardinalités, la relation devient une nouvelle table ayant pour clé primaire le couple d’identifiant des tables issues de l’association.

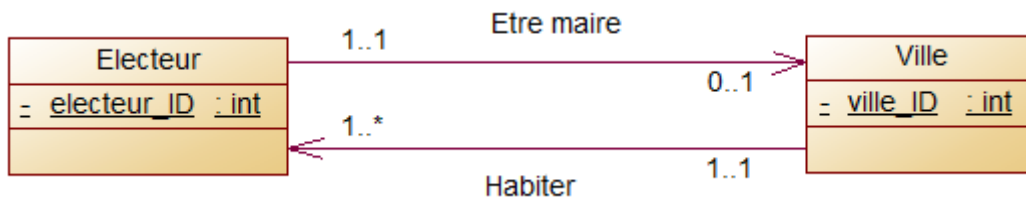
Attention : en fonction des cardinalités, la création des attributs ne sera pas la même. Par exemple, une cardinalité (1,1) donnera un attribut dont la valeur ne devra jamais être nulle et devra référencer une clé primaire d’une autre table alors qu’une cardinalité (0,1) donnera un attribut dont la valeur pourra être nulle ou devra référencer une clé primaire d’une autre table.

3. Circuits dans les dépendances référentielles

La génération du code du LDD SQL se fait à partir du schéma relationnel, ce dernier étant obtenu en appliquant pour chaque classe et pour chaque association les règles définies ci-dessus. Pour effectuer cette génération de code, on ne tient pas compte du double-sens des dépendances référentielles.

L'ordre de création des relations se fait en respectant les références en avant des dépendances référentielles. Il peut exister un (ou plusieurs circuits) dans les dépendances référentielles. Dans ce cas, il faut que le concepteur 'coupe' ce circuit, et reporte la contrainte correspondante au niveau applicatif.

Exemple :



Electeur(electeur_ID,...,residence)

Ville(ville_ID,...,lemaire)

Dans ce cas, pour pouvoir écrire le code du LDD SQL, il faut supprimer une dépendance, par exemple :

Electeur(electeur_ID,...,residence)

Ville(ville_ID,...,lemaire)

CHAPITRE V : LA DEMARCHE EN UML

U.M.L ne donne aucune démarche « normalisée »

La plus proche d'U.M.L reste objectory de Y.Jacobson, qui a donné lieu à la création d'un PROCESSUS UNIFIE.

La démarche est pilotée par les USES CASES, elle est centrée sur la création le plus tôt possible de l'architecture du logiciel, et le processus est incrémental au début du processus et Itératif dès que c'est possible.

1. Faire les cas d'utilisation.
2. Rechercher les classes et associations.
 - Analyse de texte
 - Par Use case.
3. Compléter par des diagrammes de séquence sur les uses cases.
4. Faire les Diagrammes de transition d'états.
5. En déduire les premières opérations.
6. Faire la conception du système.
 - Diagramme de déploiement.
7. Faire la conception des objets.
 - Scénario
 - Diagramme d'objets.
8. Compléter le diagramme de classes.

Le Processus est non linéaire. Il s'agit de faire les activités classiques du génie logiciel, à savoir REQUIREMENTS, ANALYSE, CONCEPTION, IMPLEMENTATION, TEST de nombreuses fois au cours du cycle qui va comprendre des phases de lancement, puis de l'élaboration, enfin de construction.

Naturellement chaque type d'activité sera privilégié et forte à son ordre. Par exemple, les requirements seront beaucoup plus important en phase de lancement qu'en phase de conception. Mais on ne peut pas nier que cela puisse arriver. En effet, certaines exigences peuvent apparaître au moment de la construction du système.